

Краткие ответы на вопросы по операционным системам

1. Введение. Что такое операционная система? Почему точное определение дать трудно? Основные функции. Назначение ОС: как расширенная машина и как менеджер ресурсов. 6
2. Введение. Состав ОС как набора подсистем. Состав ОС на примере Windows, Linux и Unix. Функции ОС. Различные классификации ОС. 8
3. Введение. Основные этапы развития ОС. Эксплуатационные требования к ОС. Что такое POSIX? Современные перспективы развития ОС. 11
4. Архитектура вычислительных машин. Что такое архитектура ВС? Типовой состав современной ВС. Основы работы процессора: определение, алгоритм работы, регистры и различные подходы к организации работы процессора. Режимы работы процессора. 13
5. Архитектура вычислительных машин. Основы памяти: определение, виды, иерархия. Связь с виртуальной памятью. Устройства ввода-вывода: компоненты и принцип их работы. Система прерываний. 15
6. Архитектура вычислительных машин. Взаимодействие компонентов ВС и интеллектуальные контроллеры. Шинная архитектура. Открытая архитектура. Различия архитектур фон Неймана и Гарварда. 17
7. Ядро операционной системы. Что такое ядро ОС? Понятие ресурса и его виды. Выделение и освобождение ресурсов. Классификация ресурсов. Объекты ядра ОС: определение, виды, структура, операции над ними. 19
8. Ядро операционной системы. Архитектура ОС. Функции ядра. Дополнительные модули ОС. Задачи решаемые ядром. Основные подсистемы. Особенности ядра в сравнении с обычными программами. 22
9. Ядро операционной системы. Ядро и режимы его работы. Виды ядер ОС по структуре. Многослойная структура ОС: слои и их назначение. Многослойная структура ядра ОС: слои и их назначение. 24
10. Процессы. Что такое процесс? Что такое многозадачность? Виды процессов. В каких случаях лучше всего прибегать к процессной многозадачности? Какие ресурсы процесса вам известны? Контекст процесса: определение, назначение и состав. 27
11. Процессы. Что такое процесс? Управление процессом: основные операции над процессами, основные состояния процессов. Понятие дочернего и родительского процессов. Функции управления на примере WinAPI и POSIX API. Особенности создания и свойства процессов. 29
12. Процессы. Что такое процесс? Межпроцессное взаимодействие (IPC): определение, классификация механизмов. Общие концепции

передачи данных: виды связей, топологии, виды обмена, буферы. В каких ситуациях предпочтительнее использовать те или иные IPC механизмы взаимодействия.....32

13. Потоки. Что такое потоки управления и выполнения? Понятие многопоточности. Связь потоков и процессов. Свойства потоков и их ресурсы. Потокобезопасность и реентерабельность. Переключение контекста и состояния потоков на примере Windows и Linux.....35

14. Потоки. Что такое потоки управления и выполнения? Типы потоков. Принципы их устройства и работы. Особенности реализации потоков на примере Windows и Linux. Что такое Thread Local Storage? Функции для работы с потоками на примере WinAPI и POSIX API.....39

15. Диспетчеризация и планирование. Понятия планирования и диспетчеризации. Процесс диспетчеризации. Планировщик: понятие, виды и суть их работы. Состояния процессов и их связь с каждым из видов планирования. События, приводящие к вызову планировщика.....43

16. Диспетчеризация и планирование. Понятия планирования и диспетчеризации. Алгоритмы планирования: определение, категории. Виды сред выполнения. Общие и специфичные задачи планирования. Примеры алгоритмов планирования для каждой из сред.....45

17. Диспетчеризация и планирование. Понятия планирования и диспетчеризации. Алгоритмы планирования: определение, виды. Алгоритмы FCFS, SJN, RR, PB: описание, преимущества и недостатки.47

18. Диспетчеризация и планирование. Понятия планирования и диспетчеризации. Алгоритмы планирования Windows и Linux: принципы работы, понятия базового приоритета и любезности. Многопроцессорное планирование. Родственное планирование. Перепланировка.....49

19. Прерывания. Система прерываний и почему она важна. Типы прерываний. Виды внутренних прерываний. Механизмы обработки прерываний. Способы выполнения прерываний. Приоритеты прерываний. Маскирование прерываний и их обработка.....51

20. Прерывания. Система прерываний и почему она важна. Программные прерывания. Планирование прерываний. Системные вызовы и их связь с прерываниями. Организация системных вызовов. Режимы системных вызовов.54

21. Синхронизация. Что такое синхронизация? Причины необходимости синхронизации. Основные термины: состояние гонки, критическая секция, критические данные, взаимное исключение. Необходимые условия решения задачи взаимного исключения. Активное и пассивное ожидание.56

22. Синхронизация. Что такое синхронизация? Типовые задачи в области синхронизации. Задачи взаимного исключения и производителя-потребителя: суть проблемы, способы решения, проблемы решения.	58
23. Синхронизация. Что такое синхронизация? Типовые задачи в области синхронизации. Задача писателя-читателя: суть проблемы, способы решения. Барьеры и события.....	60
24. Тупики. Понятие тупика. Ресурсы: понятие, выгружаемость и невыгружаемость. Ресурсная взаимоблокировка. Подходы к решению проблемы взаимоблокировок. Условия Коффмана и их особенность. Графы Холта.	62
25. Тупики. Понятие тупика. Примеры обнаружения взаимоблокировок с использованием графов Холта. Страусиный алгоритм. Метод обнаружения взаимоблокировок на основе матриц в примерах. Способы восстановления системы при взаимоблокировке: суть, сложности.	64
26. Тупики. Понятие тупика. Избегание взаимоблокировок. Безопасные и небезопасные состояния. Алгоритм банкира. Атака на условия взаимного исключения: суть, проблемы.	67
27. Компьютерное время. Что такое время? Эталонная секунда. Шкалы времени: от эфемеридного до атомного времени. UTC и атомное время. Нелинейность времени. Социальное время. Календари и часовые пояса. ...	69
28. Компьютерное время. Что такое время? Аппаратная поддержка времени. Функции часов. Понятие эпохи. Системный таймер и его виды. Часы реального времени. Взаимодействие часов реального времени и системных таймеров. Виды времени в Windows и Linux.	71
29. Управление памятью. Понятие памяти и внешней памяти. Функции операционной системы по управлению памятью. Типы адресов. Виртуальное адресное пространство: понятие, структура, назначенное и максимальное. Подходы к преобразованию адресов.	73
30. Управление памятью. Понятие памяти и внешней памяти. Понятие виртуальной памяти. Структура виртуального адресного пространства. Виды памяти и их назначение. Алгоритмы распределения памяти.	75
31. Управление памятью. Понятие памяти и внешней памяти. Алгоритмы распределения памяти без использования внешней памяти: основные понятия, преимущества и недостатки. Фрагментация.	77
32. Управление памятью. Понятие памяти и внешней памяти. Алгоритмы распределения памяти с использованием внешней памяти: виртуальная память, свопинг. Преимущества и недостатки. Виды организации виртуальной памяти и их краткое описание. Свопинг и подкачка.....	78

33. Управление памятью. Понятие памяти и внешней памяти. Алгоритмы страничного распределения памяти: страницы и их виды, таблицы страниц и их содержимое, страничные прерывания, преимущества и недостатки. Базисные свойства страничной организации. Схема преобразования адресов.	79
34. Управление памятью. Понятие памяти и внешней памяти. Оптимизация страничной памяти. Многоуровневая страничная память: принципы работы, особенности. Сегментное распределение памяти: сегменты, таблица сегментов и их содержимое, преобразование адресов, преимущества и недостатки.	81
35. Управление памятью. Понятие памяти и внешней памяти. Сегментно-страничное распределение памяти: сегменты и страницы, способы организации адресного пространства, способы адресации, алгоритмы преобразования адресов для каждого способа. Разделяемые сегменты.	83
36. Управление памятью. Виртуальная память. Стратегии выборки страниц. Стратегии размещения: виды, преимущества и недостатки. Стратегии замещения: принцип локальности, блокировка кадров, оптимальный алгоритм, преимущества и недостатки.	86
37. Управление памятью. Виртуальная память. Стратегии замещения: алгоритм дольше всех неиспользовавшегося элемента, алгоритм «первым вошел – первым вышел», часовой алгоритм, преимущества и недостатки, резидентное множество. Области видимости. Стратегии очистки.	88
38. Кэширование данных. Понятие кэша и кэширования. Цель кэширования. Свойство прозрачности. Иерархия памяти. Принципы действия кэш-памяти: понятие записи, обращение к данным и варианты развития событий. Свойства локальности. Проблема согласования данных.	90
39. Кэширование данных. Понятие кэша и кэширования. Отображение основной памяти на кэш: схемы отображения, основные понятия, особенности, смешанный подход.	92
40. Кэширование данных. Понятие кэша и кэширования. Схемы выполнения запросов в системах с кэш памятью: этапы выполнения, двухуровневый кэш. Буфер ассоциативной трансляции: понятие, назначение, принцип работы.	93
41. Управление файлами. Подсистема ввода-вывода: понятие, основные компоненты, задачи. Основы параллельной работы I/O-устройств и CPU. Проблема согласования скоростей обмена и кэширование данных. Разделение устройств и данных между процессами.	95
42. Управление файлами. Подсистема ввода-вывода. Программный интерфейс к устройствам. Поддержка широкого спектра драйверов: суть	

задачи, виды интерфейсов драйверов. Динамическая загрузка и выгрузка драйверов. Поддержка файловых систем. Синхронный и асинхронный режимы.....	97
43. Управление файлами. Подсистема ввода-вывода. Многослойная модель подсистемы ввода-вывода: вертикальный и горизонтальный слои. Менеджер ввода-вывода: понятие, функции, интерфейсы. Пример работы менеджера на примере Windows.	98
44. Управление файлами. Подсистема ввода-вывода. Понятие драйвера. Назначение драйвера. Многоуровневый драйвер: понятие, структура, взаимодействие уровней, преимущества подхода. Виды драйверов.....	99
45. Управление файлами. Файл: понятие, цели использования. Файловая система: понятие, назначение, задачи. Типы файлов. Виды иерархической структуры файловой системы. Имена файлов и их виды. Соотношения между файлами и количеством имён.	100
46. Управление файлами. Файл и файловая система. Понятие монтирования. Атрибуты файлов: виды, доступ, организация хранения. Логическая структура файла: логическая запись, структурированные и неструктурированные файловые системы, способы структуризации.	102
47. Управление файлами. Физическая организация файловой системы. Принципы работы жёсткого диска. Основные понятия: пластина, головка, дорожка, сектор. Адресация сектора. Понятие кластера и связь с секторами. Виды форматирования. Понятие раздела. Активный раздел.	104
48. Управление файлами. Физическая организация файловой системы. Критерии эффективности. Основные виды физической организации файлов: принципы, адресация, преимущества и недостатки.	106
49. Управление файлами. Физическая организация файловой системы на примерах систем FAT, ufs и s5. Основные структурные элементы и их назначение, особенности.	108
50. Управление файлами. Операции над файлами. Файловые системы с запоминанием и без запоминания состояния операций. Механизм открытия файла на примере Unix: алгоритм работы, системные структуры и их взаимосвязь. Обмен данными с файлами. Блокирование файлов.....	110

1. Введение. Что такое операционная система? Почему точное определение дать трудно? Основные функции. Назначение ОС: как расширенная машина и как менеджер ресурсов.

Что такое ОС?

В исходном материале дано несколько определений операционной системы:

1. Операционная система – комплекс системного программного обеспечения, который предоставляет полезные абстракции базовых аппаратных средств.
2. Операционная система – это комплекс программ, предназначенных для управления ресурсами компьютера и организации взаимодействия с пользователем.

Почему трудно дать точное определение?

ОС выполняет две разные функции:

- ☐ Предоставляет программистам понятные абстракции вместо сложного "железа"
- ☐ Управляет этими ресурсами

Основные функции ОС:

- ☐ Прием и обработка команд пользователя
- ☐ Загрузка и выполнение программ
- ☐ Управление памятью и виртуальной памятью
- ☐ Управление файлами и устройствами ввода-вывода
- ☐ Многозадачность (параллельное выполнение программ)
- ☐ Планирование задач
- ☐ Защита данных и обеспечение безопасности
- ☐ Организация взаимодействия между программами

ОС как расширенная машина:

ОС скрывает сложность аппаратуры. Например, вместо работы с диском на низком уровне, программист работает с файлами. ОС создает удобные абстракции (файлы, процессы) и управляет ими.

ОС как менеджер ресурсов:

ОС распределяет ресурсы (процессор, память, устройства) между программами двумя способами:

❑ **Разделение во времени:** ресурс используется по очереди (например, процессор)

❑ **Разделение в пространстве:** каждый получает часть ресурса (например, память)

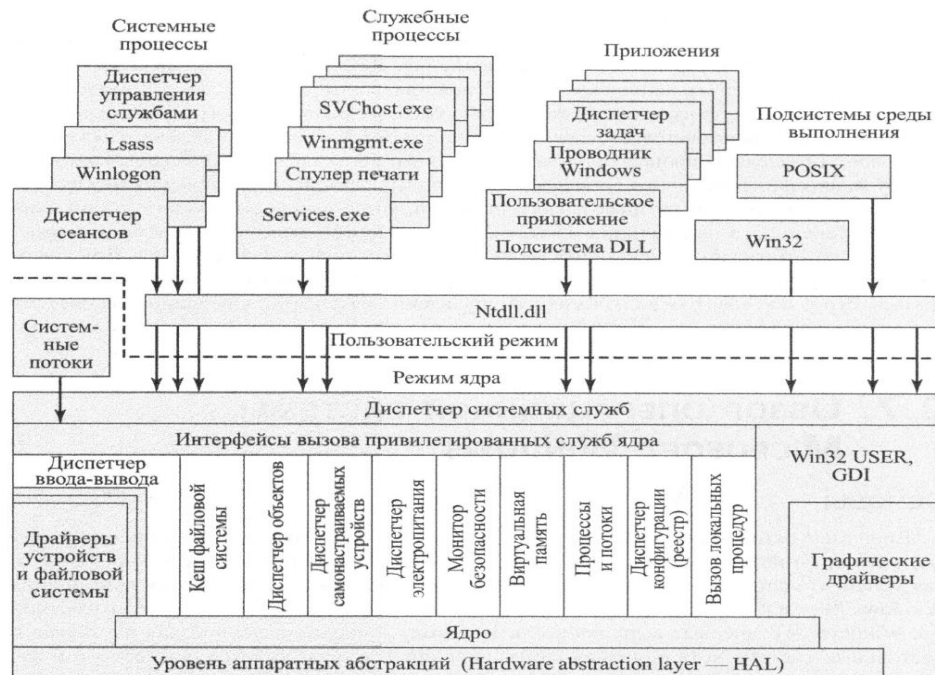
2. Введение. Состав ОС как набора подсистем. Состав ОС на примере Windows, Linux и Unix. Функции ОС. Различные классификации ОС.

Состав ОС как подсистем:

ОС состоит из подсистем, управляющих разными ресурсами:

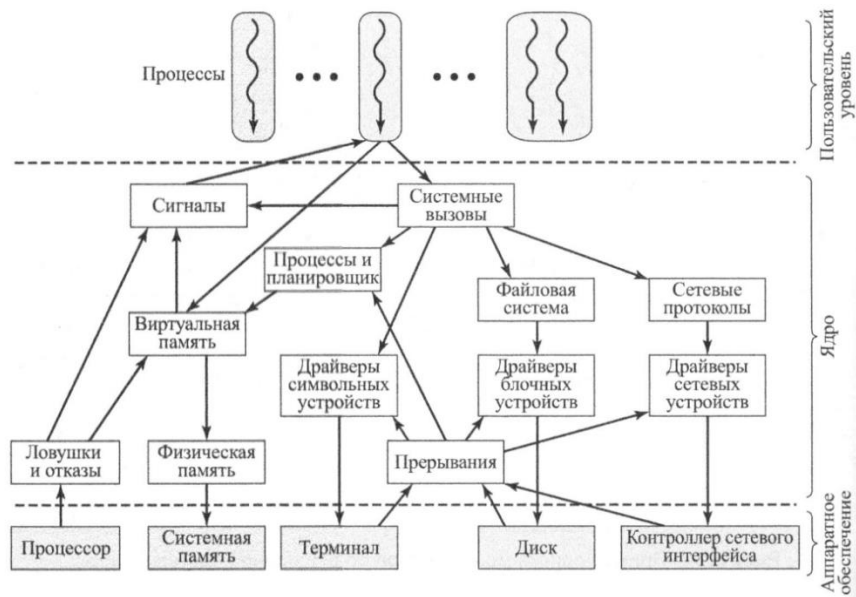
- ? Управление процессами
- ? Управление памятью
- ? Управление файлами
- ? Управление внешними устройствами
- ? Пользовательский интерфейс
- ? Защита данных и администрирование

Windows:



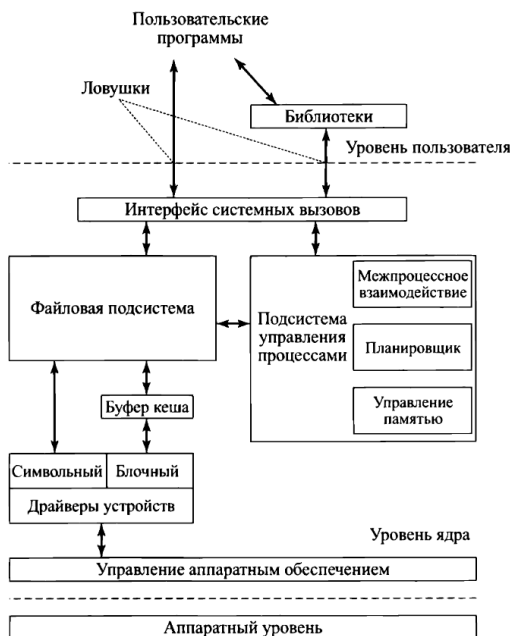
- ? **Ядро** с диспетчерами (ввода-вывода, памяти, процессов, объектов, безопасности)
- ? **HAL** (Hardware Abstraction Layer) – уровень аппаратных абстракций
- ? **Подсистемы среды выполнения:** Win32, POSIX, DLL
- ? **Системные процессы:** диспетчер служб, Lsass, Winlogon
- ? **Приложения** и системные библиотеки (Ntdll.dll)
- ? Четкое разделение пользовательского режима и режима ядра

Linux:



- ? **Ядро** координирует все подсистемы
- ? **Системные вызовы** – интерфейс между приложениями и ядром
- ? **Подсистемы:** процессы/планировщик, виртуальная память, файловая система, сетевые протоколы
- ? **Драйверы:** символьных, блочных и сетевых устройств
- ? **Прерывания и сигналы** для обработки событий

Unix:



- ? Три уровня: аппаратный, ядро, пользовательский
- ? **Интерфейс системных вызовов** – граница между ядром и пользователем
- ? **Файловая подсистема** с буфером кэша

- ❑ **Подсистема управления процессами:** планировщик, межпроцессное взаимодействие, управление памятью
- ❑ **Драйверы устройств** (символьные и блочные)
- ❑ **Библиотеки** и пользовательские программы

Классификации ОС:

- ❑ **По способу организации:** пакетная обработка, разделение времени, реального времени
- ❑ **По типу ядра:** монолитное, модульное, микроядро, экзоядро, наноядро, гибридное, комбинированное
- ❑ **По задачам:** однозадачные, многозадачные
- ❑ **По пользователям:** однопользовательские, многопользовательские
- ❑ **По процессорам:** однопроцессорные, многопроцессорные
- ❑ **По сети:** локальные, сетевые (серверные/клиентские)
- ❑ **По лицензии:** открытые, проприетарные
- ❑ **По сфере:** мэйнфреймы, серверы, ПК, мобильные, встроенные, маршрутизаторы

3. Введение. Основные этапы развития ОС. Эксплуатационные требования к ОС. Что такое POSIX? Современные перспективы развития ОС.

Этапы развития ОС:

1. **Нулевое поколение** (механические устройства): Операционных систем не было, управление вручную.
2. **Первое поколение** (1945-1955, лампы): ОС отсутствовали, программирование на машинном языке, управление оборудованием вручную.
3. **Второе поколение** (1955-1965, транзисторы): Появились системы пакетной обработки – задания собирались в пакеты и выполнялись последовательно.
4. **Третье поколение** (1965-1980, микросхемы): Многозадачность, подкачка данных, режим разделения времени (несколько пользователей через терминалы).
5. **Четвертое поколение** (1980-2005, микропроцессоры): Персональные компьютеры, дружелюбные интерфейсы (Windows, Mac OS), сетевые ОС.
6. **Пятое поколение** (2005-настоящее время): Многоядерные системы, виртуализация, мобильные ОС (Android, iOS), безопасность, облачные вычисления, IoT.

Эксплуатационные требования:

- ❑ **Расширяемость:** легко добавлять новые функции
- ❑ **Переносимость:** работа на разных платформах
- ❑ **Надежность:** защита от ошибок и сбоев
- ❑ **Совместимость:** выполнение программ для других ОС
- ❑ **Безопасность:** защита ресурсов пользователей
- ❑ **Производительность:** высокое быстродействие

POSIX:

POSIX (Portable Operating System Interface) – набор стандартов, описывающих интерфейсы между операционной системой и прикладной программой, набор стандартов для совместимости UNIX-подобных систем. Описывает системный API, библиотеку C и приложения. Обеспечивает переносимость программ.

Современные перспективы:

- ☐ Повышенное внимание к безопасности
- ☐ Многоплатформенность
- ☐ Интеллектуальные интерфейсы
- ☐ Превращение сети в виртуальный компьютер
- ☐ Квантовые вычисления

4. Архитектура вычислительных машин. Что такое архитектура ВС? Типовой состав современной ВС. Основы работы процессора: определение, алгоритм работы, регистры и различные подходы к организации работы процессора. Режимы работы процессора.

Архитектура ВС:

Архитектура вычислительной системы – это совокупность основных устройств ЭВМ и способов их взаимодействия, определяющая построение аппаратных и программных средств.

Типовой состав ВС:

- ❑ **Системный блок:** материнская плата, процессор, оперативная память, жесткий диск, видеоадаптер, звуковой адаптер, блок питания, приводы
- ❑ **Монитор, клавиатура, мышь**
- ❑ **Периферийные устройства**

Процессор (ЦПУ):

Центральное процессорное устройство (ЦПУ) – это мозг компьютера. Он выбирает команды из памяти и выполняет их.

Алгоритм работы:

1. Выборка команды из памяти
2. Декодирование (определение типа и операндов)
3. Выполнение команды
4. Повторение цикла до конца программы

Регистры процессора:

- ❑ **Счетчик команд (IP):** адрес следующей команды
- ❑ **Указатель стека (SP):** вершина стека в памяти
- ❑ **Слово состояния программы (PSW):** биты условий, приоритет, режим работы

Подходы к организации работы процессора:

- ❑ **Вычислительный конвейер:** за один такт выполняется более одной команды за счет параллельной работы узлов на разных этапах
- ❑ **Суперскалярность:** несколько одинаковых узлов выполняют команды параллельно
- ❑ **Многоядерность:** несколько физических ядер выполняют команды разных потоков параллельно

❓ **Многопоточность (гиперпоточность):** ядро сохраняет состояние двух потоков и быстро переключается между ними

Режимы работы процессора:

❓ **Режим ядра:** процессор может выполнять любые команды и использовать все возможности аппаратуры

❓ **Пользовательский режим:** выполнение только части команд, ограниченный доступ к аппаратуре

Переключение между режимами происходит через системный вызов (инструкция TRAP).

5. Архитектура вычислительных машин. Основы памяти: определение, виды, иерархия. Связь с виртуальной памятью. Устройства ввода-вывода: компоненты и принцип их работы. Система прерываний.

Память — устройство для хранения данных и программ.

Память:

Память организована в виде иерархии: верхние уровни быстрее, но меньше и дороже.

Иерархия памяти:

Выглядит подобная иерархия подобным образом:
МПП – микропроцессорная память (регистры)



Виртуальная память:

Позволяет запускать программы, превышающие физическую память, за счет размещения частей на диске. Оперативная память используется как кэш. Отображение адресов выполняет блок управления памятью (MMU).

Устройства ввода-вывода:

Состоят из двух компонентов:

- ? **Устройство:** физическое оборудование
- ? **Контроллер:** микросхема, управляющая устройством
- ? **Драйвер:** программа для общения с контроллером

Способы установки драйвера:

1. Компоновка с ядром ОС
2. Указание в конфигурационных файлах с перезагрузкой
3. Динамическая загрузка (plug and play)

Способы ввода-вывода:

1. **Системные вызовы:** драйвер опрашивает устройство (активное ожидание)

2. **Прерывания:** драйвер запускает устройство и ждет прерывания по завершении
3. **DMA (прямой доступ к памяти):** контроллер передает данные без участия процессора

Система прерываний:

Прерывание – передача управления специальной процедуре (обработчику прерываний) при наступлении события.

Виды прерываний:

- ❑ **Аппаратные:** реакция на физический сигнал от устройства
- ❑ **Программные:** вызываются командой из программы
- ❑ **Исключения:** реакция на нестандартную ситуацию в процессоре

Обработка прерываний:

1. Сохранение состояния программы, переход в режим ядра
2. Поиск и выполнение обработчика (через вектор прерываний)
3. Возврат управления программе

Особые ситуации (синхронные прерывания):

- ❑ **Ошибка (Fault):** можно исправить, продолжить с той же команды
- ❑ **Ловушка (Trap):** возникает после команды, используется для системных вызовов
- ❑ **Сбой (Abort):** некорректируемая ситуация, критическая ошибка

6. Архитектура вычислительных машин. Взаимодействие компонентов ВС и интеллектуальные контроллеры. Шинная архитектура. Открытая архитектура. Различия архитектур фон Неймана и Гарварда.

Интеллектуальные контроллеры:

Контроллер – специализированный процессор, управляющий внешним устройством по специальным встроенным программам обмена. Имеет собственную систему команд и регистры. Разгружает центральный процессор.

Шинная архитектура:

Шина – несколько проводников, соединяющих устройства.

Они могут быть внутренними по отношению к процессору и служить для передачи данных в АЛУ и из АЛУ, а могут быть внешними по отношению к процессору и связывать процессор с памятью или устройствами ввода-вывода. Состоит из трех частей:

- ❑ **Шина данных:** передача информации (8, 16, 32, 64 бит)
- ❑ **Шина адреса:** указывает адрес устройства или ячейки памяти (16, 20, 24, 32, 64 бит)
- ❑ **Шина управления:** регулирует обмен информацией через сигналы (выбор чтения/запись, синхронизация)

Также может быть шина питания (электропитание компонентов к шине).

Открытая архитектура:

Возможность легко добавлять новые устройства. Пользователь может выбирать состав устройств и конфигурировать систему. Прямые связи между устройствами (например, DMA – прямой доступ к памяти).

Современные системы имеют множество шин: кэш-памяти, памяти, PCIe, PCI, USB, SATA, DMI – каждая со своей скоростью и назначением.

Архитектура фон Неймана:

- ❑ Программы и данные хранятся в одной памяти
- ❑ Одна шина для данных и команд
- ❑ Основные блоки: АЛУ, УУ, память, устройства ввода-вывода
- ❑ Используется в персональных компьютерах
- ❑ • арифметико-логическое устройство (АЛУ), выполняющее арифметические и логические операции

- ☐ • управляющее устройство (УУ), организующее процесс выполнения программ
- ☐ • внешнее запоминающее устройство (ВЗУ), или память, для хранения программ и данных
- ☐ • оперативное запоминающее устройство (ОЗУ)
- ☐ • устройства ввода и вывода информации (УВВ)

☐

☐ **Гарвардская архитектура:**

- ☐ Программы и данные хранятся в разных запоминающих устройствах
- ☐ Раздельные шины для данных и команд
- ☐ Процессор может одновременно читать команду и работать с данными
- ☐ Используется в контроллерах и микроконтроллерах

7. Ядро операционной системы. Что такое ядро ОС? Понятие ресурса и его виды. Выделение и освобождение ресурсов. Классификация ресурсов. Объекты ядра ОС: определение, виды, структура, операции над ними.

Ядро ОС:

В исходном материале дано несколько определений ядра:

1. Ядро (kernel) – это центральная и главная часть операционной системы, которая обеспечивает архитектуру связи с приложениями, организует и регулирует доступ к ресурсам компьютера. Дополнительно, но не как правило, предоставляет доступ к сетевым протоколам и к файловой системе. Ядро операционной системы взаимодействует с приложениями пользователя, с утилитами и с системными обрабатывающими программами.

Ресурс:

В исходном материале дано несколько определений ресурса:

1. Ресурсом является любой объект, который может распределяться внутри системы. Ресурсы могут быть разделяемыми, когда несколько процессов могут их использовать одновременно (в один и тот же момент времени) или параллельно (в течение некоторого интервала времени процессы используют ресурс попеременно), а могут быть и неделимыми.

2. Термин «ресурс» обычно применяется по отношению к неоднократно используемым, относительно стабильным и «дефицитным» объектам, которые запрашиваются, используются и освобождаются процессами в период их активности.

3. В первых системах программирования под понятием «ресурсы» понимали: процессорное время, память, каналы ввода/вывода и периферийные устройства. Впоследствии понятие ресурса стало более универсальным и общим.

4. В настоящее время понятие ресурса превратилось в абстрактную структуру с целым рядом атрибутов, характеризующих способы доступа к этой структуре и ее физическое представление в системе. К ресурсам стали относиться и такие объекты, как сообщения и синхросигналы, которыми обмениваются задачи.

Выделение ресурса:

При необходимости использовать какой-либо ресурс задача обращается к супервизору ОС (центральному управляющему модулю) и сообщает о своем требовании. При этом указывается вид ресурса и, если необходимо, его объем. Затем управление передается ОС, которая:

- Переключает процессор в привилегированный режим
 - Выполняет запрос
 - Возвращает управление задаче, выдавшей данный запрос
- если ресурс занят, задача блокируется и помещается в очередь ожидания.

Освобождение ресурса:

1. Задача сообщает ОС об освобождении
2. ОС освобождает ресурс
3. Проверяет очередь и выводит задачу из состояния ожидания в состояние готовности

Классификация ресурсов:

- ❑ **По реальности:** физические (диски, принтеры) и виртуальные (RAM)
- ❑ **По расширению:** эластичные (допускают виртуализацию) и жесткие
- ❑ **По активности:** активные (ЦП) и пассивные (RAM)
- ❑ **По важности:** главные (ЦП, RAM) и второстепенные (HDD)
- ❑ **По структуре:** простые и составные
- ❑ **По восстанавливаемости:** воспроизводимые (запрос-использование-освобождение) и потребляемые (освобождение-запрос-использование)
- ❑ **По использованию:** последовательные (принтер), параллельные (данные в RAM), критически последовательные (буфер)

Объекты ядра ОС:

В исходном материале дано несколько определений объекта ядра:

Объект ядра – это коллекция данных, являющихся частью режима ядра операционной системы, которыми управляет Диспетчер объектов Windows (Windows Object Manager).

В рамках ядра Linux разработчики явно не используют определение «объект ядра». Вместо этого акцент делается не на формальном определении объектов ядра, а на практической реализации механизмов управления ресурсами и взаимодействия с аппаратурой.

Объекты ядра используются системой и приложениями для управления множеством разных ресурсов: процессами, потоками, файлами и т.д. Приложение не может напрямую обращаться к объектам ядра, читать и изменять их содержимое.

Виды объектов ядра:

- ❑ Объекты устройств
- ❑ Объекты файлов
- ❑ Символические ссылки
- ❑ Разделы реестра
- ❑ Потоки и процессы
- ❑ Объекты диспетчера (события, мьютексы)
- ❑ Объекты обратного вызова
- ❑ Объекты section

Структура объекта:

Блок памяти с информацией об объекте. Содержит дескриптор защиты, счетчик пользователей и специфичные для типа объекта данные.

Операции над объектами:

- ❑ Создание (CreateXxx) с именем, атрибутами защиты
- ❑ Открытие (OpenXxx)
- ❑ Работа через описатель
- ❑ Удаление при обнулении счетчика пользователей

8. Ядро операционной системы. Архитектура ОС. Функции ядра. Дополнительные модули ОС. Задачи решаемые ядром. Основные подсистемы. Особенности ядра в сравнении с обычными программами.

Архитектура ОС:

ОС делится на модули:

- Модули, выполняющие основные функции ОС – ядро ОС
- Модули, выполняющие вспомогательные функции ОС
- Модули ядра выполняют базовые функции ОС:
 - Управление процессами
 - Управление памятью
 - Управление устройствами ввода-вывода

Функции ядра:

1. **Внутрисистемные функции:** переключение контекстов, загрузка страниц, обработка прерываний (недоступны приложениям)
2. **Функции поддержки приложений:** образуют API, обслуживают системные вызовы

Свойства функций ядра:

- ☐ Скорость выполнения определяет производительность всей системы
- ☐ Модули резидентны (постоянно в памяти)

Дополнительные модули ОС:

- ☐ **Утилиты:** программы управления и сопровождения (сжатие, дефрагментация)
- ☐ **Библиотеки:** процедуры и функции для упрощения разработки
- ☐ **Программы дополнительных услуг:** калькулятор, игры
- ☐ **Системные обрабатывающие программы:** редакторы, компиляторы, отладчики

Обычно загружаются в память только на время выполнения (транзитные).

Задачи ядра:

- ☐ Синхронизация процессов и обмен данными
- ☐ Выполнение ввода-вывода
- ☐ Управление памятью
- ☐ Поддержка файловых систем

- ❑ Обеспечение вызова и возврата процедур
- ❑ Обработка прерываний

Основные подсистемы ядра:

- ❑ **Управление процессами:** создание/удаление, распределение ресурсов, синхронизация, взаимодействие
- ❑ **Управление памятью:** распределение памяти, перемещение процессов на диск (подкачка)
- ❑ **Файловая подсистема:** унифицированный интерфейс доступа к данным, контроль прав доступа
- ❑ **Ввод/вывод:** выполнение запросов на доступ к устройствам, взаимодействие с драйверами

Особенности ядра:

- ❑ **Режим работы:** привилегированный режим (приложения – пользовательский)
- ❑ **Резидентность:** постоянно в памяти
- ❑ **Доступ к памяти:** ядро имеет доступ ко всей памяти, но защищено от приложений
- ❑ **Системные вызовы:** приложения обращаются через системные вызовы
- ❑ **Производительность:** скорость определяет производительность системы
- ❑ **Переключение режимов:** системный вызов переключает процессор в привилегированный режим

Особенности ядра Linux:

- ❑ Нет доступа к стандартным библиотекам C
- ❑ Нет защиты памяти
- ❑ Нет замещения страниц (каждый байт = байт физической памяти)
- ❑ Нельзя использовать вычисления с плавающей точкой
- ❑ Фиксированный стек (не рекомендуется рекурсия)
- ❑ Переносимость (платформо-независимый код)

9. Ядро операционной системы. Ядро и режимы его работы. Виды ядер ОС по структуре. Многослойная структура ОС: слои и их назначение. Многослойная структура ядра ОС: слои и их назначение.

Режимы работы ядра:

Ядро работает в **привилегированном режиме**, приложения – в **пользовательском режиме**. Это обеспечивает защиту: приложения не могут выполнять опасные инструкции и обращаться к памяти ОС.

Многослойная структура ОС: слои и их назначение:

Многослойная структура ОС – это подход к организации операционной системы, при котором система делится на несколько уровней (слоёв), каждый из которых предоставляет сервисы вышележащему слою и использует сервисы нижележащего.

Основные слои ОС (снизу вверх):

1. Аппаратный слой:

- ☐ Физическое оборудование компьютера (процессор, память, устройства ввода-вывода)
- ☐ Обеспечивает базовые аппаратные функции

2. Слой ядра ОС:

- ☐ Центральная часть операционной системы
- ☐ Управляет процессами, памятью, устройствами ввода-вывода
- ☐ Работает в привилегированном режиме
- ☐ Обеспечивает базовые механизмы работы системы

3. Слой системных вызовов (API):

- ☐ Интерфейс между ядром и приложениями
- ☐ Предоставляет стандартизированные функции для доступа к ресурсам системы
- ☐ Обеспечивает защиту и контроль доступа

4. Слой системных библиотек:

- ☐ Библиотеки функций, упрощающие работу с системными вызовами
- ☐ Предоставляют высокоуровневые интерфейсы для приложений
- ☐ Примеры: стандартные библиотеки языка C, библиотеки ввода-вывода

5. **Слой приложений:**

- ☐ Пользовательские программы и системные утилиты
- ☐ Работают в пользовательском режиме
- ☐ Используют сервисы нижележащих слоёв через системные вызовы и библиотеки

Виды ядер по структуре:

1. **Монолитное ядро:**

- ☐ Все компоненты в едином адресном пространстве
- ☐ Нет деления на слои и модули
- ☐ Большая скорость, простота конструирования
- ☐ Недостаток: ошибка в любом элементе может заблокировать всю систему

2. **Модульное ядро:**

- ☐ Модифицированная версия монолитного
- ☐ Возможность подгрузки модулей (драйверов) динамически или статически
- ☐ Не требует перестройки ядра при изменении оборудования

3. **Микроядро:**

- ☐ Решает только простые задачи управления процессами
- ☐ Большая часть функций в процессах-сервисах пользователя
- ☐ Преимущества: переносимость, расширяемость, надежность, поддержка распределенных вычислений
- ☐ Недостаток: снижение производительности из-за накладных расходов

4. **Экзоядро:**

- ☐ Предоставляет только обмен между процессами и распределение ресурсов

5. **Наноядро:**

- ☐ Очень простое ядро, решает только обработку аппаратных прерываний

6. **Гибридное ядро:**

- ☐ Объединяет преимущества монолитного и микроядерного подходов
- ☐ Высокая производительность, изоляция подсистем

7. **Комбинированное ядро:**

- ❑ Комбинация вышеперечисленных вариантов

Многослойная структура ядра ОС:

1. Средства аппаратной поддержки ОС:

- ❑ Аппаратура, участвующая в организации вычислительного процесса
- ❑ Привилегированный режим, прерывания, защита памяти

2. Машинно-зависимые компоненты:

- ❑ Модули, отражающие специфику аппаратной платформы
- ❑ Экранируют вышележащие слои от особенностей «железа» (предоставление единого интерфейса)

3. Базовые механизмы ядра:

- ❑ Примитивные операции ядра
- ❑ Исполняют решения, принятые верхними слоями

4. Менеджеры ресурсов:

- ❑ Управление процессами, вводом-выводом, файловой системой, памятью
- ❑ Стратегические задачи по управлению ресурсами

5. Интерфейс системных вызовов:

- ❑ Верхний слой ядра
- ❑ Взаимодействует с приложениями, образует API

Количество слоев может варьироваться в разных системах.

10. Процессы. Что такое процесс? Что такое многозадачность? Виды процессов. В каких случаях лучше всего прибегать к процессной многозадачности? Какие ресурсы процесса вам известны? Контекст процесса: определение, назначение и состав.

Что такое процесс?

В исходном материале дано несколько определений процесса:

1. Процесс – управляющий объект, который обеспечивает изоляцию адресных пространств и представляет работающий экземпляр программы.
2. Процесс – исполняемое на устройстве приложение вместе со всеми ресурсами, которые требуются для его исполнения.

Что такое многозадачность?

Многозадачность (мультипрограммирование) – постоянное переключение процессора между процессами. Процессор быстро переключается между процессами (десятки-сотни миллисекунд каждому), создавая иллюзию параллельной работы, хотя в каждый момент работает только с одним процессом.

Виды процессов:

- ❑ **Системные процессы:** управляют ресурсами в интересах прикладных процессов (распределение памяти, обмен страницами, контроль устройств). Выполняют служебные задачи ОС.
- ❑ **Пользовательские процессы:** выполняют задачи пользователя
- ❑ **Демоны (Unix) / Службы (Windows):** фоновые процессы для обработки активной деятельности

Когда использовать процессную многозадачность:

- ❑ Независимые задачи, требующие изоляции и безопасности
- ❑ Ресурсоёмкие задачи и задачи с вводом/выводом
- ❑ Когда важна устойчивость к ошибкам

Ресурсы процесса:

- ❑ Образ исполняемого файла
- ❑ Память
- ❑ Список дескрипторов объектов (файлы, потоки, объекты синхронизации)

- ❑ Атрибуты безопасности (маркеры)
- ❑ Контекст процесса
- ❑ Идентификатор процесса (PID)
- ❑ Идентификатор родительского процесса (PPID)
- ❑ Состояние процесса
- ❑ Флаги процесса
- ❑ Информация для диспетчеризации

Контекст процесса:

В исходном материале дано несколько определений контекста процесса:

Контекст процесса включает:

- ❑ **Регистровый контекст** – содержимое регистров процессора (включая счётчик команд)
- ❑ **Системный контекст** – системные регистры процессора
- ❑ **Пользовательский контекст** – данные в адресном пространстве процесса

Контекст процесса сохраняет состояние процесса для переключения между процессами и возобновления работы.

5. Контекст процесса – минимальный набор данных, используемый процессом, который должен быть сохранен, чтобы выполнение процесса могло быть прервано и в последующем возобновлено с той же точки.

Назначение:

Контекст процесса необходим для сохранения состояния процесса при его переключении, чтобы можно было вернуться к его выполнению с того же места.

Состав контекста:

- ❑ Program Counter (PC) или Instruction Pointer (IP)
- ❑ Значений регистров
- ❑ Стека
- ❑ Кучи
- ❑ Глобальных переменных
- ❑ И т.д.

11. Процессы. Что такое процесс? Управление процессом: основные операции над процессами, основные состояния процессов. Понятие дочернего и родительского процессов. Функции управления на примере WinAPI и POSIX API. Особенности создания и свойства процессов.

Что такое процесс?

В исходном материале дано несколько определений процесса:

2. Процесс – управляющий объект, который обеспечивает изоляцию адресных пространств и представляет работающий экземпляр программы.
3. Процесс – исполняемое на устройстве приложение вместе со всеми ресурсами, которые требуются для его исполнения.

Основные операции над процессами:

- ❑ **Запуск процесса:** ОС выбирает один процесс из состояния готовности для исполнения.
- ❑ **Приостановка процесса:** Работа процесса в состоянии исполнение приостанавливается в результате прерывания.
- ❑ **Блокирование процесса:** Процесс блокируется, когда не может продолжать работу, ожидая возникновения события.
- ❑ **Разблокирование процесса:** После возникновения события ОС определяет, какое событие произошло, и разблокирует соответствующий процесс.
- ❑ **Переключение контекста:** Сохранение состояния одного процесса и загрузка состояния другого.

Основные состояния процессов:

- ❑ **Состояние рождения:** Первый этап жизненного цикла, происходит полное формирование процесса как системной сущности.
- ❑ **Состояние готовности:** Процесс полностью подготовлен к выполнению, но ожидает своей очереди на процессор в очереди планировщика.
- ❑ **Состояние ожидания:** Процесс не может продолжить выполнение из-за внешних факторов.
- ❑ **Состояние исполнения:** Процесс получает возможность использовать процессорное время, выполняет свои задачи.

❓ **Состояние завершения:** Процесс прекращает существование, все ресурсы освобождаются, информация о завершении передается родительскому процессу.

Дочерний и родительский процессы:

❓ **Дочерний процесс** – процесс, создаваемый в результате системного вызова.

❓ **Родительский процесс** – процесс, инициировавший системный вызов.

Функции управления на примере WinAPI и POSIX API:

Назначение	WinAPI	POSIX
Создание процесса	CreateProcessA (ANSI), CreateProcessW (UTF-8)	fork, exec
Ожидание завершения процесса	WaitForSingleObject, WaitForMultipleObjects	wait
Завершение процесса	ExitProcess (штатное), TerminateProcess (аварийное), CloseHandle	exit
Получение PID и PPID	GetCurrentProcessId, GetProcessId	getpid, getppid
Переменные окружения	GetEnvironmentVariable, SetEnvironmentVariable	getenv, putenv, setenv, unsetenv
Текущий диск и каталог	GetCurrentDirectory, SetCurrentDirectory	getcwd, chdir

Создание процессов:

Windows:

Поиск файла: каталог EXE → текущий каталог → системные каталоги → PATH.
Если указан полный путь – используется напрямую.

Linux:

❓ **fork** – создает копию процесса

- ❑ **exec** – заменяет программу в процессе
- ❑ Поиск: если путь содержит "/" – используется как путь, иначе – поиск в PATH

Свойства процесса:

- ❑ Соответствует исполняемому файлу
- ❑ Имеет PID и PPID (родитель)
- ❑ Изолирован от других процессов
- ❑ Имеет адресное пространство (code, data, heap, stack)
- ❑ Имеет контекст (сохраняется при переключении)
- ❑ Имеет стандартные потоки: ввод (0), вывод (1), ошибки (2)
- ❑ Единица работы ОС

12. Процессы. Что такое процесс? Межпроцессное взаимодействие (IPC): определение, классификация механизмов. Общие концепции передачи данных: виды связей, топологии, виды обмена, буферы. В каких ситуациях предпочтительнее использовать те или иные IPC механизмы взаимодействия.

Что такое процесс?

В исходном материале дано несколько определений процесса:

- ❑ Процесс – управляющий объект, который обеспечивает изоляцию адресных пространств и представляет работающий экземпляр программы.
- ❑ Процесс – исполняемое на устройстве приложение вместе со всеми ресурсами, которые требуются для его исполнения.

Межпроцессное взаимодействие (IPC):

IPC (межпроцессное взаимодействие) – механизм, позволяющий процессам обмениваться данными и синхронизировать свои действия.

Классификация механизмов IPC:

Выделяются три уровня средств IPC:

- ❑ **Локальный уровень:** Средства привязаны к процессору, возможны только в пределах компьютера. К этому виду относятся каналы, разделяемая память и очереди сообщений.
- 2. **Удаленный уровень:** Механизмы обеспечивают взаимодействие как в пределах одного процессора, так и между программами на различных процессорах через сеть. Сюда относятся удаленные вызовы процедур (RPC), сокеты Unix, TLI (Transport Layer Interface).
- 3. **Высокоуровневый уровень:** Пакеты программного обеспечения, реализующие промежуточный слой между системной платформой и приложением.

Общие концепции передачи данных:

Виды связей по направлению:

- 4. **Симплексная:** передача только в одном направлении

- 5. **Полудуплексная:** передача в обе стороны, но одновременно только в одну сторону
- 6. **Дуплексная:** передача в двух направлениях одновременно

Топология связи:

Топология связи – конфигурация связей между процессами-отправителями и адресатами. Различают:

- 5. 1-1 (один к одному)
- 6. 1-N (один ко многим)
- 7. N-1 (многие к одному)
- 8. N-M (многие ко многим)

Виды обмена:

Обмен данными выполняется при помощи двух функций:

- 6. **send** – послать данные
- 7. **receive** – получить данные

Адресация процессов:

- 7. **Прямая адресация:** в функциях send и receive явно указываются процессы отправитель и адресат
- 8. **Косвенная адресация:** указываются не адреса, а имя связи, по которой передаются данные
- 9. **Симметричная адресация:** используется только прямая или только косвенная адресация
- 10. **Асимметричная адресация:** используется как прямая, так и косвенная адресация

Буферы:

Буфер – количество данных которые могут одновременно пересылаться по этой связи.

Типы буферизации:

- ☐ **Нулевая вместимость (нет буфера):** возможен только синхронный обмен данными
- ☐ **Ограниченная вместимость (ограниченный буфер):** если буфер полон, отправитель должен ждать очистки буфера
- ☐ **Неограниченная вместимость (неограниченный буфер):** отправитель никогда не ждет при отправке сообщения

В каких ситуациях использовать IPC механизмы:

Разделяемая память:

- ☐ Дуплексный и синхронный обмен
- ☐ Любая топология связей
- ☐ Используется для: высокой производительности, больших объёмов данных, гибкой работы с данными

Передача данных:

- ☐ Используется для: слабосвязанных процессов, децентрализованных систем, масштабирования, устойчивости к сбоям, упрощённой синхронизации

13. Потоки. Что такое потоки управления и выполнения? Понятие многопоточности. Связь потоков и процессов. Свойства потоков и их ресурсы. Потокобезопасность и реентерабельность. Переключение контекста и состояния потоков на примере Windows и Linux.

Потоки управления и выполнения:

5. **Поток управления** – порядок выполнения команд программы (логическая последовательность)

☐ **Поток выполнения** – реальное исполнение кода процессором. С точки зрения ОС – объект ядра, которому выделяется процессорное время. Наименьшая единица работы для ядра ОС.

Многопоточность:

Многопоточность – возможность системы поддерживать работу множества потоков в рамках одного процесса.

Связь потоков и процессов:

В рамках процесса могут находиться один или несколько потоков, каждый из которых обладает:

- ☐ Состоянием выполнения потока (выполняющийся, готовый к выполнению и т.д.)
- ☐ Сохраненным контекстом не выполняющегося потока (независимый счетчик команд)
- ☐ Стеком выполнения
- ☐ Статической памятью для локальных переменных
- ☐ Доступом к памяти и ресурсам процесса (разделяется всеми потоками процесса)

Все потоки процесса разделяют между собой состояние и ресурсы процесса. Они находятся в одном и том же адресном пространстве и имеют доступ к одним и тем же данным. Если один поток изменяет данные в памяти, другие потоки могут отследить эти изменения.

Свойства потоков и их ресурсы:

У каждого потока имеется свой **TCB (Thread Control Block)** – структура данных ядра ОС, содержащая информацию специфичную для потоков, необходимую для управления потоком. TCB является "манифестацией потока в операционной системе".

Информация в TCB:

- ❑ TID (идентификатор потока)
- ❑ Указатель на стек
- ❑ Программный счетчик (текущая инструкция этого потока)
- ❑ Состояние потока
- ❑ Регистры потока
- ❑ Указатель на PCB (Process Control Block)

Потокобезопасность и реентерабельность:

1. **Потокобезопасная функция** – функция, которая может быть вызвана одновременно из нескольких потоков без риска возникновения ошибок или непредсказуемого поведения.

❑ **Реентерабельная функция** (повторно входимая) – функция, которую можно безопасно вызывать из нескольких потоков одновременно. Должна:
Не использовать изменяемые глобальные переменные (которые могут меняться другими потоками)

Не использовать статические переменные внутри функции

Не возвращать указатель на статические данные внутри функции

Переключение контекста:

Переключение контекста (context switch) – это когда процессор переключается с одного процесса (потока) на другой. При этом сохраняется состояние текущего процесса (регистры, счётчик команд и т.д.), чтобы потом можно было продолжить его выполнение с того же места, и загружается состояние нового процесса.

Потоки, как и процессы, имеют свой собственный контекст.

Переключение контекста на примере Windows и Linux:

Windows:

- 4. Выполняется диспетчером ядра (Kernel Dispatcher)
- 5. Сохраняются регистры процессора, стек, состояние потока
- 6. Используются структуры KTHREAD и ETHREAD для хранения контекста
- 7. Переключение происходит при:
Вытеснении потоком более высокого приоритета

Истечении кванта времени

Блокировке потока (ожидание события, синхронизация)

Добровольной передаче управления (yield)

Linux:

4. Выполняется планировщиком ядра (Scheduler)
5. Сохраняются регистры процессора, стек, состояние процесса/потока
6. Используется структура task_struct для хранения контекста
7. В Linux потоки реализованы как процессы, разделяющие адресное пространство (LWP - Lightweight Processes)
8. Переключение происходит при:
 - Прерывании таймера
 - Системном вызове
 - Блокировке процесса/потока
 - Добровольной передаче управления

Действия, связанные с изменением состояния потоков:

❑ **Порождение** – создание нового потока с собственным контекстом (регистры, стек)

4. **Блокирование** – поток переходит в ожидание события (сохраняется его контекст)

❑ **Разблокирование** – поток переходит в готовность при наступлении ожидаемого события

❑ **Завершение** – удаление контекста потока после завершения работы

Состояния потоков в Windows:

❑ **Готовый к выполнению:** Поток может быть направлен на выполнение. Диспетчер ядра отслеживает все готовые потоки и осуществляет их планирование.

❑ **Резервный:** Поток, который будет запущен следующим на данном процессоре. Если приоритет достаточно высок, может вытеснить выполняющийся поток.

❑ **Выполняющийся:** Поток пребывает в этом состоянии до вытеснения, истечения кванта времени, блокировки или завершения.

- ❑ **Ожидающий:** Поток входит в это состояние, если блокирован событием, ждет синхронизации или приостановлен средой подсистемы.
- ❑ **Завершенный:** Поток завершен и удаляется из системы.

Состояния потоков в Linux:

2. **Выполняющийся:** Текущий процесс либо выполняется, либо готов к выполнению.
3. **Прерываемый:** Состояние блокировки, в котором процесс ожидает наступления события (завершение I/O, освобождение ресурса, сигнал).
4. **Непрерываемый:** Состояние блокировки, в котором процесс ожидает выполнения аппаратного условия и не воспринимает сигналы.
5. **Остановленный:** Процесс был остановлен и может быть продолжен только при воздействии другого процесса.
6. **Зомби:** Процесс был прекращен, но его структура остается в таблице процессов.

14. Потоки. Что такое потоки управления и выполнения? Типы потоков. Принципы их устройства и работы. Особенности реализации потоков на примере Windows и Linux. Что такое Thread Local Storage? Функции для работы с потоками на примере WinAPI и POSIX API.

Потоки управления и выполнения:

2. **Поток управления** – логическая последовательность выполнения команд программы. Это просто порядок, в котором выполняются команды, определяющий маршрут выполнения программы.
3. **Поток выполнения** – независимая последовательность инструкций в рамках процесса, которая отвечает за реальное исполнение кода программы.

Типы потоков:

Обычно выделяют две категории потоков:

4. **Пользовательские потоки (user-level threads – ULT):** Все действия по управлению потоками выполняются самим приложением; ядро не подозревает о существовании потоков. Используется специальная библиотека для работы с потоками. **Устройство потока:**

5. Каждый поток имеет собственный **контекст выполнения** (регистры процессора, программный счётчик, стек)
6. Поток использует **TCB (Thread Control Block)** – структуру данных ядра для хранения информации о потоке
7. Все потоки процесса **разделяют адресное пространство** процесса (код, данные, открытые файлы)
8. Каждый поток имеет **собственный стек** для локальных переменных и вызовов функций

Принцип работы:

4. **Создание потока:**

Выделяется память для стека потока

Создаётся TCB с контекстом потока

Поток добавляется в очередь готовых потоков

Планировщик может выбрать поток для выполнения

5. **Выполнение потока:**

Планировщик выбирает поток из очереди готовых
Загружается контекст потока (регистры, программный счётчик)
Поток получает квант времени процессора
Выполняются инструкции потока

❑ **Переключение потоков:**

Сохраняется контекст текущего потока в TCB
Выбирается следующий поток из очереди
Загружается контекст нового потока
Выполнение продолжается с точки останова нового потока

❑ **Блокировка потока:**

При ожидании события (I/O, синхронизация) поток переходит в состояние ожидания
Контекст сохраняется
Управление передаётся другому потоку
При наступлении события поток возвращается в очередь готовых

3. **Завершение потока:**

Поток завершает выполнение функции
Освобождаются ресурсы потока (стек, TCB)
Уведомляются ожидающие потоки (если есть)

Преимущества ULT:

Переключение потоков не предусматривает переход в режим ядра (меньше накладных расходов)
Планирование может выполняться с учетом специфики приложения
Может работать в любой операционной системе

Недостатки ULT:

При блокирующем системном вызове блокируется весь процесс
Не может воспользоваться преимуществами многопроцессорной системы (ядро закрепляет за каждым процессом только один процессор)

❑2. **Потоки на уровне ядра (kernel-level threads – KLT):** Все действия по управлению потоками выполняются ядром. Ядро поддерживает информацию контекста процесса и каждого потока. Планирование выполняется ядром на основе потоков.

1. Преимущества KLT:

Ядро может одновременно планировать работу нескольких потоков одного процесса на нескольких процессорах

При блокировке одного потока ядро может выбрать для выполнения другой поток этого же процесса

Процедуры ядра могут быть многопоточными

2. Недостаток KLT:

Для передачи управления от одного потока другому приходится переключаться в режим ядра

3. Комбинированный подход: Создание потоков выполняется в пользовательском пространстве, несколько потоков на пользовательском уровне отображаются в такое же или меньшее число потоков на уровне ядра. Сочетает преимущества обоих подходов.

Особенности реализации потоков:

Windows:

Потоки являются объектами ядра и представлены структурами KTHREAD и ETHREAD.

Linux:

Linux не признает различие между потоками и процессами. Пользовательские потоки отображаются на процессы уровня ядра. Несколько потоков пользовательского уровня отображаются на процессы уровня ядра Linux, которые разделяют один и тот же идентификатор группы. В Linux новый процесс создается путем копирования атрибутов текущего процесса.

Thread Local Storage (TLS):

Thread Local Storage – механизм, который позволяет каждому потоку хранить свои собственные данные в отдельном пространстве, которое не доступно другим потокам. Это позволяет каждому потоку иметь свои собственные значения переменных, которые не будут перезаписаны другими потоками. Задействование данных, относящихся к отдельному потоку, позволяет сделать функцию потокобезопасной, не изменяя при этом ее интерфейс.

Функции для работы с потоками:

Создание потока:

- ❑ Windows: CreateThread, _beginthreadex
- ❑ POSIX: pthread_create

Завершение потока:

- ❑ Windows: ExitThread, TerminateThread, _endthreadex
- ❑ POSIX: pthread_exit, pthread_cancel

Способы завершения потока в Windows:

2. Функция потока возвращает управление (рекомендуемый способ)
3. Поток самоуничтожается вызовом ExitThread (нежелательный способ)
4. Один из потоков вызывает TerminateThread (нежелательный способ)
5. Завершается процесс, содержащий данный поток (нежелательно)

Способы завершения потока в POSIX:

3. Начальная функция выполняет return, указывая возвращаемое значение
4. Поток вызывает pthread_exit
5. Поток отменяется с помощью pthread_cancel
6. Любой из потоков вызывает exit или главный поток выполняет return в main (завершает все потоки)

Управление потоком:

4. Windows: SuspendThread, ResumeThread, WaitForSingleObject, WaitForMultipleObjects, GetExitCodeThread, GetCurrentThread
5. POSIX: pthread_join (ожидает завершения потока), pthread_detach (отсоединяет поток), pthread_self (возвращает идентификатор текущего потока), pthread_once (гарантирует выполнение кода только один раз)

15. Диспетчеризация и планирование. Понятия планирования и диспетчеризации. Процесс диспетчеризации. Планировщик: понятие, виды и суть их работы. Состояния процессов и их связь с каждым из видов планирования. События, приводящие к вызову планировщика.

Понятия планирования и диспетчеризации:

Диспетчеризация – процесс переключения между процессами/потоками: сохранение состояния текущего, выбор следующего через планировщика, восстановление состояния нового процесса.

Планирование – процесс выбора следующего процесса/потока для выполнения из готовых. Направлено на эффективное распределение ресурсов процессора.

Процесс диспетчеризации:

5. Формирование контекста прерываемого процесса
6. Восстановление состояния нового процесса (загрузка регистров)
7. Загрузка адреса следующей команды в счётчик команд

Планировщик:

Часть ОС, отвечающая за выбор следующего процесса/потока для выполнения.

Виды планировщиков:

6. **Долгосрочное:** Решение о добавлении нового процесса к активным. Определяет степень многозадачности. Используется в пакетных системах.
2. **Среднесрочное:** Часть системы свопинга. Решение о загрузке процесса в память. Связано с управлением памятью.
3. **Краткосрочное (диспетчер):** Определяет, какой процесс выполняется следующим. Вызывается при событиях, требующих перепланирования.

Состояния процессов и их связь с планированием:

4. **Долгосрочное:** Новый → Готовый (управляет степенью многозадачности)
5. **Среднесрочное:** Готовый ↔ Выгруженный (управляет свопингом)
6. **Краткосрочное:** Готовый ↔ Выполняющийся ↔ Ожидающий

События, приводящие к вызову планировщика:

Долгосрочное: При создании нового процесса.

Краткосрочное:

5. Прерывание таймера (истёк квант времени)
6. Прерывания ввода-вывода (блокировка процесса)
7. Системные вызовы (освобождение ресурса)
8. Внешние прерывания (завершение I/O)
9. Внутренние прерывания (ошибки)

16. Диспетчеризация и планирование. Понятия планирования и диспетчеризации. Алгоритмы планирования: определение, категории. Виды сред выполнения. Общие и специфичные задачи планирования. Примеры алгоритмов планирования для каждой из сред.

Алгоритмы планирования:

Метод управления распределением процессорного времени между процессами/потоками. Цель – оптимизация работы системы.

Категории:

6. **Приоритетные (вытесняющие):** Процесс работает до истечения кванта времени, затем вытесняется. Требуется прерываний по таймеру.
7. **Неприоритетные (кооперативные):** Процесс выполняется до блокировки или добровольного освобождения CPU.

Виды сред выполнения:

7. **Пакетная:** Нет требований к быстрому ответу. Приемлемы неприоритетные алгоритмы или приоритетные с длительными периодами.
8. **Интерактивная:** Важна приоритетность для предотвращения захвата CPU одним процессом.
9. **Реального времени:** Процессы выполняются быстро, приоритетность иногда не требуется.

Задачи планирования:

Общие (для всех систем):

8. **Равнодоступность (беспристрастность)** - все процессы получают справедливую долю CPU, без голодания
9. **Принуждение к политике** - выполнение установленных правил планирования
10. **Баланс загрузки системы** - равномерная загрузка всех компонентов системы

Пакетные системы:

2. **Производительность (заданий/час)** - максимум заданий в час
3. **Оборотное время (от передачи до завершения)** - минимизация времени от подачи задания до завершения
4. **Использование CPU** - максимальная загрузка процессора

Интерактивные системы:

3. Время отклика (минимизация) - минимизация задержки между действием пользователя и ответом системы
4. Пропорциональность (приоритет интерактивным запросам) - интерактивные запросы получают приоритет для быстрого ответа

Системы реального времени:

4. Предельный срок (максимизация процессов, завершающихся в срок) - максимум процессов завершается до дедлайна
5. Предсказуемость (одинаковое время выполнения) - время выполнения стабильно, независимо от загрузки системы

Примеры алгоритмов:

Пакетные: FCFS, SJN, SRT

Интерактивные: RR, Priority, SPN, MLFQ, Lottery, Fair-share

Реального времени:

5. **Rate Monotonic (RM):** Приоритет по периоду (короткий период = высокий приоритет)
6. **Deadline Monotonic (DM):** Приоритет по дедлайну (ранний дедлайн = высокий приоритет)

17. Диспетчеризация и планирование. Понятия планирования и диспетчеризации. Алгоритмы планирования: определение, виды. Алгоритмы FCFS, SJN, RR, PB: описание, преимущества и недостатки.

Алгоритмы планирования:

Метод управления распределением процессорного времени между процессами/потоками.

Виды:

6. **Приоритетные (вытесняющие):** Процесс работает до истечения кванта, затем вытесняется
7. **Неприоритетные (кооперативные):** Процесс выполняется до блокировки или добровольного освобождения CPU

Алгоритм FCFS (First Come – First Served):

Неприоритетный алгоритм "первым пришел – первым обслужен". CPU выделяется в порядке поступления запросов.

Преимущества: Простота, понятность, справедливость, предсказуемость

Недостатки: Эффект конвоя, отсутствие приоритетов, высокое время ожидания, низкая эффективность в интерактивных системах

Алгоритм SJN (Shortest Job Next):

Невытесняющий алгоритм для пакетных систем. Выбирается самое короткое задание (требует знания времени выполнения).

Преимущества: Минимизация среднего времени ожидания, эффективность с короткими задачами

Недостатки: Проблема голодания длинных процессов, необходимость предсказания времени выполнения

Алгоритм RR (Round Robin):

Циклическое планирование с квантом времени. Процесс работает квант, затем помещается в конец очереди.

Преимущества: Высокая отзывчивость, справедливость, отсутствие голодания

Недостатки: Критичность выбора кванта, накладные расходы на переключение контекста, не учитывает приоритеты

Алгоритм PB (Priority Based):

Запускается процесс с наивысшим приоритетом из готовых.

Преимущества: Простота, гибкость, гарантированное выполнение высокоприоритетных задач

Недостатки: Проблема голодания, высокие затраты в динамических системах, зависимость от настройки приоритетов

18. Диспетчеризация и планирование. Понятия планирования и диспетчеризации. Алгоритмы планирования Windows и Linux: принципы работы, понятия базового приоритета и любезности. Многопроцессорное планирование. Родственное планирование. Перепланировка.

Алгоритмы планирования Windows и Linux:

Windows:

2. Единица планирования – поток
3. Приоритеты: потоков и процессов
4. Уровни приоритета: 0-31 (0 только для потока обнуления страниц)
5. Кванты времени назначаются циклически потокам с наивысшим приоритетом
6. Высокоприоритетный поток вытесняет низкоприоритетный

Базовый приоритет:

Определяется классом приоритета процесса и уровнем приоритета потока (значения 1-31).

Linux:

3. Не различает поток и процесс (потоки = процессы, разделяющие адресное пространство)
4. Приоритет определяет долю процессорного времени

Любезность (nice):

- ❑ Значение nice: -20 (высокий) до +19 (низкий)
- ❑ Приоритет реального времени: 0-99
- ❑ Обычный пользователь может только уменьшать приоритет
- ❑ Приоритеты nice (-20...+19) соответствуют приоритетам планировщика (100...139)

Многопроцессорное планирование:

Два измерения: какой поток запускать? На каком процессоре?

Два подхода

- ❑ **Родственное планирование (affinity):** Поток выполняется на том же процессоре, где запускался в последний раз
- ❑ Единая структура данных (общая очередь)

Перепланировка:

События, требующие перераспределения CPU:

- ☐ **(T) Прерывание таймера** – истёк квант времени
- ☐ **(I/O) Системный вызов I/O** – блокировка процесса
- ☐ **(R) Освобождение ресурса** – пробуждение ожидающих процессов
- ☐ **(S) Внешнее прерывание** – завершение I/O операции
- ☐ **(ER) Внутреннее прерывание** – ошибка выполнения

19. Прерывания. Система прерываний и почему она важна. Типы прерываний. Виды внутренних прерываний. Механизмы обработки прерываний. Способы выполнения прерываний. Приоритеты прерываний. Маскирование прерываний и их обработка.

Система прерываний:

Механизм управления обработкой событий. Останавливает выполнение текущей программы и переключается на обработку события, требующего немедленного внимания.

Важность:

- ☐ Реакция на события в реальном времени
- ☐ Эффективное использование процессора (вместо опроса устройств)
- ☐ Параллельная обработка задач
- ☐ Реализация многозадачности
- ☐ Обработка ошибок и исключений
- ☐ Высокая производительность

Типы прерываний:

6. **Внешние (аппаратные):** От устройств, пользователя, датчиков. Электрический сигнал от аппаратуры.

☐ **Внутренние (исключения):** Синхронно с выполнением программы при аварийной ситуации (деление на ноль, ошибки защиты памяти, несуществующий адрес, привилегированная инструкция в пользовательском режиме).

3. **Программные:** При выполнении специальной команды процессора (INT, trap, syscall), имитирующей прерывание.

Виды внутренних прерываний:

1. **Отказы (faults):** Выявляются до выполнения инструкции (страничный отказ). После обработки команда повторяется.

2. **Ловушки (traps):** Обрабатываются после выполнения инструкции (переполнение). Выполняется следующая инструкция.

3. **Аварийные завершения (aborts):** Невозможно определить команду, вызвавшую прерывание (серьёзные сбои аппаратуры).

Механизмы обработки:

- ❑ Приоритезация прерываний
- ❑ **IRQ (interrupt request)** – уведомление о прерывании
- ❑ **ISR (Interrupt Service Routine)** – обработчик прерывания
- ❑ Аппаратные прерывания → драйверы устройств
- ❑ Исключения → модули ядра
- ❑ Программные прерывания → процедуры системных вызовов
- ❑ **Диспетчер прерываний** – координирует обработчики

Способы выполнения прерываний:

1. **Векторные:** Передаётся уровень приоритета и адрес обработчика. Вектор прерывания (фиксированный/конфигурируемый/программируемый) идентифицирует обработчик.
2. **Опрашиваемые (polled):** Передаётся только уровень приоритета. Процессор вызывает все обработчики уровня, пока один не подтвердит прерывание.

Приоритеты прерываний:

Приоритезация: Источники делятся на классы с уровнями приоритета.

Обслуживание:

- ❑ **Относительные приоритеты:** Выбирается высший приоритет, но процедура не откладывается при появлении более приоритетных запросов
- ❑ **Абсолютные приоритеты:** Более приоритетные прерывания могут вытеснять менее приоритетные

Маскирование прерываний:

Временное запрещение прерываний любого класса независимо от приоритета.

Последовательность обработки:

1. Аппаратное распознавание типа прерывания
2. Сохранение контекста прерванного потока
3. Загрузка адреса обработчика и режима работы процессора

4. Временный запрет прерываний данного типа
5. Восстановление контекста и возобновление работы

20. Прерывания. Система прерываний и почему она важна. Программные прерывания. Планирование прерываний. Системные вызовы и их связь с прерываниями. Организация системных вызовов. Режимы системных вызовов.

Программные прерывания:

Возникают при выполнении специальной команды процессора (INT, trap, syscall, TISS), имитирующей прерывание. Процессор выполняет ту же последовательность, что и при внешнем/внутреннем прерывании, но в предсказуемой точке программы.

Планирование прерываний:

ОС должна упорядочивать обработчики прерываний так же, как планировщик упорядочивает пользовательские потоки. Планировщик потоков сам вызывается по прерываниям.

Механизм:

- ☐ Приоритетные очереди для обработчиков прерываний
- ☐ **Диспетчер прерываний** – координирует обработчики

Работа диспетчера:

1. Вызывается первым при прерывании
2. Запрещает прерывания на время
3. Выясняет причину
4. Сравнивает приоритет прерывания с текущим приоритетом
5. Приостанавливает текущий обработчик или помещает новый в очередь

Системные вызовы:

Позволяют приложению обратиться к ОС для выполнения действий ядра.

Требования:

- ☐ Переключение в привилегированный режим
- ☐ Высокая скорость вызова
- ☐ Единообразное обращение для всех платформ
- ☐ Легкое расширение набора
- ☐ Контроль корректности использования

Реализуются через программные прерывания (единственный способ переключения режима на большинстве платформ).

Организация:

Централизованная схема с диспетчером системных вызовов:

1. Приложение выполняет программное прерывание с фиксированным номером вектора
2. Передаёт номер системного вызова (индекс в таблице sysent)
3. Диспетчер вызывает соответствующую процедуру ОС

Режимы системных вызовов:

Обеспечивают переключение пользовательский → привилегированный режим для операций:

- ☐ Управление процессами/потоками
- ☐ Управление памятью
- ☐ Доступ к I/O устройствам
- ☐ Управление файловой системой
- ☐ Сетевые операции

После выполнения управление возвращается в пользовательский режим.

21. Синхронизация. Что такое синхронизация? Причины необходимости синхронизации. Основные термины: состояние гонки, критическая секция, критические данные, взаимное исключение. Необходимые условия решения задачи взаимного исключения. Активное и пассивное ожидание.

Синхронизация:

Процесс согласования скоростей потоков путём приостановки одного или нескольких до наступления события и последующей активизации.

Причины необходимости:

3. Доступ к общим данным (избежание порчи)
4. Согласованность состояния (правильный порядок операций)
5. Предотвращение гонок
6. Безопасность ресурсов
7. Координация работы потоков

Основные термины:

Состояние гонки: Ситуация, когда несколько потоков читают/записывают общие данные, и результат зависит от относительного времени выполнения потоков.

Критическая секция: Часть программы, результат которой может непредсказуемо меняться, если переменные изменяются другими потоками во время выполнения.

Критические данные: Данные, к которым обращается критическая секция. Их несогласованное изменение ведёт к ошибкам.

Взаимное исключение: Гарантия, что в каждый момент времени только один поток выполняет критическую секцию, работающую с общим ресурсом.

Необходимые условия решения:

4. Два потока не могут одновременно находиться в критических областях
5. Не должно быть предположений о скорости/количестве процессоров

6. Потоки вне критических областей не блокируются другими
7. Потоки не должны вечно ждать входа в критические области

Активное ожидание (busy waiting):

Поток крутится в цикле, проверяя переменную для получения разрешения. Используется в спинлоках, когда ожидание непродолжительное.

Пассивное ожидание:

Поток приостанавливается ОС и освобождает процессор до наступления события. Не тратит ресурсы, применяется при длительном ожидании.

22. Синхронизация. Что такое синхронизация? Типовые задачи в области синхронизации. Задачи взаимного исключения и производителя-потребителя: суть проблемы, способы решения, проблемы решения.

Синхронизация:

Процесс согласования скоростей потоков путём приостановки до наступления события.

Типовые задачи:

- ☐ Взаимное исключение
- ☐ Производитель-потребитель
- ☐ Читатели-писатели
- ☐ Клиент-официант
- ☐ Обедающие философы

Задача взаимного исключения:

Суть: Гарантировать, что только один поток выполняет критическую секцию с общим ресурсом.

Способы решения:

- ☐ Аппаратные: отключение прерываний, специальные команды
- ☐ Программные: мьютексы, атомарные операции, спинлоки

Проблемы: Deadlock, starvation

Задача производителя-потребителя:

Суть: Два потока используют общий буфер фиксированного размера. Производитель помещает данные, потребитель извлекает. Потоки имеют разные скорости.

Решение: Семафоры Дейкстры.

Примитивы:

- ☐ **V(S):** Увеличение S на 1 (атомарно)
- ☐ **P(S):** Уменьшение S на 1, если возможно; иначе ожидание (атомарно)

Решение задачи:

- ☐ **e** – число пустых ячеек (начальное: N)
- ☐ **f** – число заполненных ячеек (начальное: 0)

Поток-производитель: $P(e) \rightarrow \text{запись} \rightarrow V(f)$

Поток-потребитель: $P(f) \rightarrow \text{чтение} \rightarrow V(e)$

Преимущества семафоров:

- ☐ Гибкость (до N потоков)
- ☐ Эффективная синхронизация доступа к ресурсным пулам

Двоичный семафор: Семафор со значениями 0 и 1 (блокирующая переменная).

Проблемы: Deadlock, starvation

23. Синхронизация. Что такое синхронизация? Типовые задачи в области синхронизации. Задача писателя-читателя: суть проблемы, способы решения. Барьеры и события.

Синхронизация:

Процесс согласования скоростей потоков путём приостановки до наступления события.

Типовые задачи:

☐ Взаимное исключение, производитель-потребитель, читатели-писатели, клиент-официант, обедающие философы

Задача читателей-писателей:

Суть: Моделируется доступ к базе данных. Требования:

☐ Несколько потоков могут одновременно читать

☐ При записи никакой другой поток не может получить доступ (даже для чтения)

Решение: RW-локи (Readers–Writer Lock)

RW-локи решают задачу читателей-писателей: несколько потоков могут читать одновременно, а при записи доступ эксклюзивный. Это примитив синхронизации с двумя режимами: shared (для чтения) и exclusive (для записи). Читатели получают shared-блокировку и работают параллельно, пока нет писателя. Писатель получает exclusive-блокировку: во время записи никто не читает и не пишет. Если есть читатели, писатель ждёт их завершения; если пишет писатель, читатели и другие писатели ждут. Порядок предоставления блокировки не гарантируется (не FIFO). RW-локи эффективны, когда чтение частое, а запись редкая, так как чтение не изменяет данные и может выполняться параллельно, а запись требует эксклюзивного

События (Windows):

Суть: Примитив синхронизации с двумя состояниями (сигнальное/несигнальное). Позволяет вручную переводить в сигнальное состояние.

Типы:

❓ **Сброс вручную:** Остаётся сигнальным до явного сброса (ResetEvent). Освобождает любое количество потоков.

❓ **Автоматический сброс:** Автоматически сбрасывается после освобождения одного потока.

- Создание объекта события – CreateEvent
- Ожидание сигнального состояния – WaitForSingleObject
- Получение существующего объекта по имени – OpenEvent
- Установка события в сигнальное состояние – SetEvent
- Установка несигнального состояния – ResetEvent

Барьеры:

Суть: Синхронизация фаз выполнения. Ни один поток не переходит к следующей фазе, пока все потоки не завершат текущую.

Применение: Параллельные вычисления (например, итерации над матрицей), где следующая итерация требует завершения предыдущей всеми потоками.

Функции для работы с барьерами:

Windows:

3. InitializeSynchronizationBarrier – инициализация барьера
4. EnterSynchronizationBarrier – вход в барьер (ожидание всех потоков)
5. DeleteSynchronizationBarrier – удаление барьера

POSIX:

4. pthread_barrier_init – инициализация барьера
5. pthread_barrier_wait – ожидание у барьера (все потоки должны вызвать)
6. pthread_barrier_destroy – уничтожение барьера

❓

24. Тупики. Понятие тупика. Ресурсы: понятие, выгружаемость и невыгружаемость. Ресурсная взаимоблокировка. Подходы к решению проблемы взаимоблокировок. Условия Коффмана и их особенность. Графы Холта.

Понятие тупика:

Взаимная блокировка – ситуация, когда два и более потоков не могут работать, так как каждый ожидает действия другого процесса.

Ресурсы:

Понятие: Объекты с исключительным доступом (устройства, записи данных, файлы). Должны предоставляться, использоваться и высвобождаться.

Выгружаемость и невыгружаемость:

Выгружаемые: Могут быть безболезненно отобраны у процесса (например, память). Взаимоблокировки с такими ресурсами устраняются перераспределением.

Невыгружаемые: Нельзя отобрать без сбоя вычислений (например, принтер в процессе печати, пишущий привод). Во взаимоблокировках обычно фигурируют невыгружаемые ресурсы.

Последовательность использования: Запрос → Использование → Высвобождение

Ресурсная взаимоблокировка:

Каждый процесс из группы ожидает ресурса, которым обладает другой процесс из той же группы. Никто не может работать, высвободить ресурс или возобновить работу.

Подходы к решению:

7. **Игнорирование** (страусиный алгоритм)
8. **Предотвращение** – не позволять одному из условий возникновения выполниться
9. **Устранение** – не выполнять запрос, если может привести к взаимоблокировке
10. **Обнаружение** – периодическая проверка и восстановление

Условия Коффмана:

Для возникновения взаимоблокировки должны выполняться все четыре условия:

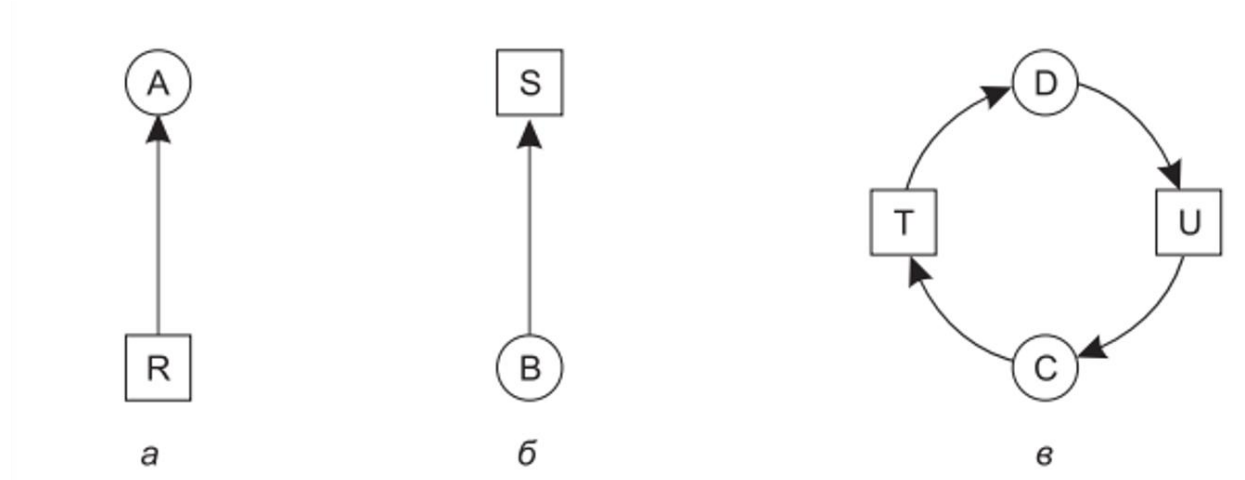
9. **Взаимное исключение:** Ресурс выделен одному процессу или доступен
10. **Удержание и ожидание:** Процесс с ресурсом может запрашивать новые
11. **Невыгружаемость:** Ресурсы нельзя принудительно отобрать
12. **Циклическое ожидание:** Кольцевая последовательность процессов, ожидающих ресурсы друг друга

Особенность: Если одно условие не соблюдается, взаимоблокировка невозможна.

Графы Холта:

Направленные графы для моделирования условий:

8. Узлы: процессы (окружности), ресурсы (квадраты)
9. Рёбра: запросы и выделения ресурсов



Используются для обнаружения взаимоблокировок.

25. Тупики. Понятие тупика. Примеры обнаружения взаимоблокировок с использованием графов Холта. Страусиный алгоритм. Метод обнаружения взаимоблокировок на основе матриц в примерах. Способы восстановления системы при взаимоблокировке: суть, сложности.

Понятие тупика:

Взаимная блокировка – ситуация, когда два и более потоков не могут работать, так как каждый ожидает действия другого процесса.

Графы Холта:

Используются для обнаружения взаимоблокировок через моделирование запросов и выделений ресурсов в виде направленных графов.

Процесс P1: владеет R1, запрашивает R2

Процесс P2: владеет R2, запрашивает R3

Процесс P3: владеет R3, запрашивает R1

$R1 \rightarrow P1 \rightarrow R2 \rightarrow P2 \rightarrow R3 \rightarrow P3 \rightarrow R1$

Страусиный алгоритм:

Игнорирование проблемы взаимоблокировок. Математики считают неприемлемым, инженеры – приемлемым, если взаимоблокировки редки (например, раз в 5 лет при еженедельных сбоях по другим причинам).

Метод обнаружения на основе матриц:

Использует матрицы для отслеживания запросов, выделений и доступных ресурсов для обнаружения взаимоблокировок.

Три матрицы: матрица выделения и матрица запросов (ресурсов на процессы) Считываем вектор доступных ресурсов: Всего ресурса - суммы столбца ресурса матрицы выделения

Потом сравниваем каждый запрос с вектором доступных ресурсов, если не больше то можем завершить и складываем вектор выделения под этот ресурс с вектором доступных ресурсов

Если нет то не можем завершить ресурс

Рекурсивно проделываем это все.

Система:

- 2 процесса: P0, P1
- 2 типа ресурсов: R0, R1
- Всего ресурсов: R0=1, R1=1

Матрица выделения (Allocation):

	R0	R1
P0	1	0
P1	0	1

Матрица запросов (Request):

	R0	R1
P0	0	1
P1	1	0

Вектор доступных (Available):

R0	R1
0	0

Проверяем процессы:

P0: Request[0] = [0, 1] ≤ Available [0, 0]? НЕТ (1 > 0)
→ Не может завершиться

P1: Request[1] = [1, 0] ≤ Available [0, 0]? НЕТ (1 > 0)
→ Не может завершиться

```
P0 владеет R0, запрашивает R1
P1 владеет R1, запрашивает R0
↓
P0 ждет R1 (у P1)
P1 ждет R0 (у P0)
↓
ЦИКЛ: P0 → R1 → P1 → R0 → P0
↓
DEADLOCK!
```

Способы восстановления:

11. **Прекращение процессов:** Принудительное завершение процессов из взаимоблокировки. Освобождает ресурсы.

❑ **Откат процессов:** Возврат к состоянию до запроса ресурсов. Требует контрольных точек (checkpoints).

❑ **Отбор ресурсов:** Принудительное изъятие ресурсов (только выгружаемых).

Сложности:

- ❑ Выбор процесса для прекращения (приоритет, выполненная работа, ресурсы)
- ❑ Потеря работы при прекращении
- ❑ Накладные расходы на откат (хранение состояний)
- ❑ Отбор возможен только для выгружаемых ресурсов
- ❑ Риск повторной взаимоблокировки
- ❑ Высокая стоимость восстановления

26. Тупики. Понятие тупика. Избегание взаимоблокировок. Безопасные и небезопасные состояния. Алгоритм банкира. Атака на условия взаимного исключения: суть, проблемы.

Понятие тупика:

Взаимоблокировка в группе процессов, где каждый ожидает события, зависящего от другого процесса группы.

Избегание взаимоблокировок:

Система не выполняет запрос ресурса, если его выделение может привести к взаимоблокировке. Используются алгоритмы проверки безопасности состояния.

Безопасное состояние:

Существует порядок планирования, при котором каждый процесс может завершиться, даже если все запрасят максимум ресурсов.

Небезопасное состояние:

Нет гарантии завершения всех процессов. Не является взаимоблокировкой, но может к ней привести.

Алгоритм банкира:

Проверяет, ведёт ли запрос к небезопасному состоянию. Если да – запрос отклоняется. Модель банкира с клиентами-кредитами.

Атака на условия взаимного исключения:

Устранение ресурсов в единоличном пользовании (например, спулинг для принтера).

Диск: [Процесс 1: 50%][Процесс 2: 50%] = 100% заполнен

Процесс 1: Заполнил 50% → Нужно еще место → БЛОКИРОВАН

Процесс 2: Заполнил 50% → Нужно еще место → БЛОКИРОВАН

Демон: Ждет завершения данных → Ждет (но не блокирует)

Процесс 1 ждет освобождения места (у Процесса 2)

Процесс 2 ждет освобождения места (у Процесса 1)

↓

ЦИКЛ: Процесс 1 → Диск → Процесс 2 → Диск → Процесс 1

↓

DEADLOCK!

Проблемы: Не все ресурсы можно сделать разделяемыми; спулинг создаёт новый ресурс (диск), что может привести к взаимоблокировке.

27. Компьютерное время. Что такое время? Эталонная секунда. Шкалы времени: от эфемеридного до атомного времени. UTC и атомное время. Нелинейность времени. Социальное время. Календари и часовые пояса.

Что такое время:

Физическая величина, одна из основных в Международной системе единиц (СИ). Используется для измерения длительности событий.

Эталонная секунда:

9 192 631 770 периодов излучения атома цезия-133. Определение введено в 1967 году, основано на атомных часах.

Шкалы времени:

ЕТ (Эфемеридное время):

Равномерная шкала, основанная на орбитальном движении Земли вокруг Солнца. Не зависит от скорости вращения Земли. Использовалось с 1960 по 1984 год, затем заменено динамическим временем.

DT (Динамическое время):

- 8. **TDТ** – для наблюдений с Земли, отнесено к центру масс Земли
- 9. **TDB** – для расчётов Солнечной системы, отнесено к барицентру (центру масс Солнечной системы)

ТТ (Земное время):

Современный астрономический стандарт для наблюдений с Земли. Учитывает эффекты общей теории относительности. Основано на атомных часах. ТТ опережает TAI на 32,184 секунды.

TAI (Международное атомное время):

Основано на атомных часах (цезий-133). Самое точное и непрерывное время. Средневзвешенное значение более 450 атомных часов в 80+ лабораториях. Точность до $2 \cdot 10^{-12}$. Основа для UTC.

UT (Всемирное время):

Основано на вращении Земли. **UT1** – основная версия, вычисляется по углу вращения Земли относительно квазаров. Неравномерно из-за замедления вращения Земли.

UTC (Всемирное координированное время):

Атомное время (основано на TAI) с корректировками високосными секундами для приближения к UT1. Международный стандарт для часовых поясов. Прерывистая шкала (корректируется целыми секундами). По состоянию на 2017 год UTC отстаёт от TAI на 37 секунд.

Нелинейность времени:

Вращение Земли неравномерно и замедляется. Это вызывает расхождение между TAI (атомное) и UT1 (по вращению). Для синхронизации UTC с UT1 добавляются високосные секунды, что делает UTC нелинейной шкалой.

Социальное время:

Восприятие длительности событий человеком, не совпадающее с физическим временем. Зависит от психологических факторов и важности событий.

Календари:

8. **Юлианский:** Год = 365,25 суток (високосный каждые 4 года). Накопление ошибки.
9. **Григорианский:** Год = 365,2425 суток. Високосный: делится на 4, но не на 100, или делится на 400. Используется в большинстве стран.

Часовые пояса:

- ☐ **Географический:** Теоретический пояс 15° по долготе (каждый пояс ± 1 час)
- ☐ **Административный:** Установленное законом время (может не совпадать с географическим)
- ☐ **UTC ± 0 :** Стандартное время (Гринвич). Остальные пояса: UTC $\pm N$
- ☐ **DST (летнее время):** Сезонный перевод часов на ± 1 час для экономии энергии

28. Компьютерное время. Что такое время? Аппаратная поддержка времени. Функции часов. Понятие эпохи. Системный таймер и его виды. Часы реального времени. Взаимодействие часов реального времени и системных таймеров. Виды времени в Windows и Linux.

Аппаратная поддержка времени:

Часы (таймеры):

- ☐ Простейшие – от сети 50/60 Гц
 - ☐ Программируемые – кварцевый генератор, счетчик, регистр хранения
- Режим однократного импульса
Режим прямоугольного импульса (clock ticks)

Функции часов:

- ☐ Ведение времени суток
- ☐ Не даёт одному процессу захватить процессор надолго
- ☐ Подсчёт времени, которое процесс использует процессор
- ☐ Обработка системного вызова alarm
- ☐ Сторожевые таймеры для ОС(Если система не отвечает, таймер может инициировать перезагрузку)
- ☐ Статистика

Эпоха:

Момент отсчёта времени. Примеры:

- ☐ Unix: 01.01.1970 00:00:00 UTC
- ☐ NT Time: 01.01.1601
- ☐ FAT: 01.01.1980

Системный таймер: Аппаратный компонент, генерирующий периодические прерывания для ОС? Используется для отсчёта времени, планирования процессов и синхронизации

Системные таймеры:

- ☐ **PIT** (Programmable Interval Timer) – исторический для x86
- ☐ **HPET** (High Precision Event Timer) – с 2005. Используется в современных системах для точных измерений времени
- ☐ **APIC таймер** – встроенный в каждое ядро процессора. Используется для локальных прерываний на каждом ядре

RTC (Real Time Clock):

Часы реального времени с автономным питанием. Учитывают текущее время, дату, день недели.

Взаимодействие RTC и системных таймеров:

- ☐ При включении RTC предоставляет начальное время
- ☐ ОС инициализирует системные часы
- ☐ Аппаратный таймер генерирует прерывания, ОС увеличивает счётчик тиков
- ☐ ОС периодически синхронизируется с RTC

Время в Windows:

- ☐ Системное (UTC) – GetSystemTime/SetSystemTime
- ☐ Локальное – GetLocalTime/SetLocalTime
- ☐ Файловое (100 нс интервалы с 01.01.1601) – GetFileTime
- ☐ Время Windows (мс с запуска) – GetTickCount
- ☐ Время прерываний (100 нс с запуска)

Время в Linux (POSIX):

- ☐ Календарное (time_t, struct tm)
- ☐ Функции: time(), localtime(), gmtime(), mktime(), strftime()
- ☐ Таймеры POSIX.1b: CLOCK_REALTIME, CLOCK_MONOTONIC, CLOCK_PROCESS_CPUTIME_ID, CLOCK_THREAD_CPUTIME_ID

29. Управление памятью. Понятие памяти и внешней памяти. Функции операционной системы по управлению памятью. Типы адресов. Виртуальное адресное пространство: понятие, структура, назначенное и максимальное. Подходы к преобразованию адресов.

Понятие памяти:

- 6. **Память** – оперативная память (RAM), требует электропитания
- 7. **Внешняя память** – жёсткий диск, файл подкачки

Функции ОС по управлению памятью:

- 7. Отслеживание свободной и занятой памяти
- 8. Выделение и освобождение памяти процессам
- 9. Своппинг (выгрузка элементов процесса на диск и потом загрузить)
- 10. Настройка адресов программы на область физической памяти
- 11. Защита данных
- 12. Дефрагментация

Типы адресов:

- 11. **Символьные имена** – присваивает пользователь
- 12. **Виртуальные (логические)** – вырабатывает транслятор (от нуля)
- 13. **Физические** – реальные номера ячеек памяти

Виртуальное адресное пространство:

- 13. Совокупность виртуальных адресов процесса

Структура:

- 10. Системная часть (одинакова для всех процессов) Содержит код и данные ядра ОС
- 11. Пользовательская часть (индивидуальна) Содержит код и данные программы пользователя

Назначенное виртуальное адресное пространство:

Адреса, действительно нужные процессу. Назначается транслятором, может расширяться.

Максимальное виртуальное адресное пространство:

Ограничено разрядностью адреса, не совпадает с объёмом физической памяти.

Подходы к преобразованию адресов:

12. **Перемещающий загрузчик** – замена виртуальных адресов физическими один раз при загрузке
13. **Динамическое преобразование** – при каждом обращении к памяти ($VA \rightarrow PA$ через смещение S)

30. Управление памятью. Понятие памяти и внешней памяти. Понятие виртуальной памяти. Структура виртуального адресного пространства. Виды памяти и их назначение. Алгоритмы распределения памяти.

Понятие виртуальной памяти:

Механизм, который позволяет процессу использовать больше памяти, чем есть физически в компьютере

Структура виртуального адресного пространства:

Системная и пользовательская части (обычно по 2 ГБ в Windows NT).

Виды памяти:

- ❑ **Куча (heap)** – динамическая память, выделяется постранично
- ❑ **Стек (stack)** – для параметров функций, локальных переменных, адреса возврата
- ❑ **Пул памяти** – paged (выгружаемый пул) Память ядра, которую можно выгружать на диск (swap) и non-paged (невыгружаемый пул) Память ядра, которую нельзя выгружать на диск
- ❑ **Свободная виртуальная память** — неиспользуемые виртуальные адреса, доступные для выделения процессам
- ❑ **Резервные списки** — списки свободных отсортированных страниц физической памяти

Алгоритмы распределения памяти:

Без внешней памяти:

- ❑ **Физическими разделами** – память разбита на области фиксированного размера
Простота, но ограничен уровень мультипрограммирования
- ❑ **Динамические разделы** – выделение по запросу
Гибкость, но фрагментация
- ❑ **Перемещаемые разделы** – сжатие для устранения фрагментации
Эффективнее, но требует времени

С внешней памятью:

- ❑ **Страничное** – перемещение страницами фиксированного размера
- ❑ **Сегментное** – перемещение сегментами произвольного размера
- ❑ **Сегментно-страничное** – комбинация обоих подходов

31. Управление памятью. Понятие памяти и внешней памяти. Алгоритмы распределения памяти без использования внешней памяти: основные понятия, преимущества и недостатки. Фрагментация.

Память:

- 10. **Память** – оперативная память (RAM), требует электропитания
- 11. **Внешняя память** – жёсткий диск, файл подкачки (swap file)

Алгоритмы без внешней памяти:

1. Фиксированные разделы:

- 10. Память разбита на области фиксированного размера
- 11. **Преимущества:** Простота реализации
- 12. **Недостатки:** Жёсткость, ограничен уровень мультипрограммирования, независимо от размера программы занимает весь раздел

2. Динамические разделы:

- ☐ Память не делится заранее, выделяется по запросу
- ☐ **Функции ОС:** Ведение таблиц свободных/занятых областей, выбор раздела (первый/наименьший/наибольший)
- ☐ **Преимущества:** Большая гибкость
- ☐ **Недостатки:** Фрагментация памяти

3. Перемещаемые разделы:

- ☐ Перемещение занятых участков для создания единой свободной области (сжатие)
- ☐ **Преимущества:** Эффективное использование памяти
- ☐ **Недостатки:** Требуется время, необходим динамический перевод адресов

Фрагментация:

Появление множества несмежных участков свободной памяти маленького размера, хотя суммарный объём достаточен.

32. Управление памятью. Понятие памяти и внешней памяти. Алгоритмы распределения памяти с использованием внешней памяти: виртуальная память, свопинг. Преимущества и недостатки. Виды организации виртуальной памяти и их краткое описание. Свопинг и подкачка.

Виртуальная память и свопинг:

- ❑ **Свопинг** – выгрузка образов процессов целиком
- ❑ **Виртуальная память** – перемещение частей (сегментов, страниц) образов процессов
- ❑ Свопинг – частный случай виртуальной памяти

Преимущества виртуальной памяти:

- ❑ Выполнение процессов с виртуальным пространством > физической памяти
- ❑ Эффективное использование памяти (загрузка только нужных частей)
- ❑ Выгрузка только части образа

Недостатки свопинга:

- ❑ Избыточность (загрузка всего образа)
- ❑ Неспособность загрузить процесс с виртуальным пространством > свободной памяти

Виды организации виртуальной памяти:

- ❑ **Страничная** – перемещение страницами фиксированного размера
- ❑ **Сегментная** – перемещение сегментами произвольного размера
- ❑ **Сегментно-страничная** – деление на сегменты, а сегменты делится на страницы

Свопинг и подкачка:

- ❑ Страничный файл = область/файл свопинга = файл подкачки
- ❑ Временное хранение сегментов и страниц
- ❑ Размер страничного файла влияет на количество одновременных приложений которые могут выполнить ОС
- ❑ Рекомендуемый размер: 1.5-2х от размера RAM

33. Управление памятью. Понятие памяти и внешней памяти. Алгоритмы страничного распределения памяти: страницы и их виды, таблицы страниц и их содержимое, страничные прерывания, преимущества и недостатки. Базисные свойства страничной организации. Схема преобразования адресов.

Страницы:

- ❑ **Виртуальные страницы** – части виртуального адресного пространства фиксированного размера
- ❑ **Физические страницы (кадры)** – части оперативной памяти того же размера
- ❑ Размер страницы = степень двойки (512, 1024, 4096 байт)

Таблицы страниц:

- 8. Создаётся для каждого процесса
- 9. Содержит информацию о том, где находится каждая виртуальная страница процесса
- 10. Используется для преобразования виртуального адреса в физический

Страничные прерывания:

- 13. Возникают при обращении к странице, отсутствующей в памяти
- 14. Процесс блокируется
- 15. Страница загружается с диска, если нет свободных страниц – выбор страницы для выгрузки

Преимущества:

- 14. Устранение внешней фрагментации
- 15. Поддержка виртуальной памяти
- 16. Защита памяти
- 17. Разделение страниц между процессами

Недостатки:

- 14. Внутренняя фрагментация (последняя страница)
- 15. Накладные расходы на таблицы страниц
- 16. Страничные прерывания замедляют работу
- 17. Сложность реализации (MMU)

Базисные свойства:

- 12. Размер страницы = 2^k (степень двойки)

13. Смещения в виртуальном и физическом адресах равны ($sv = sf$)

Схема преобразования адресов:

14. Из регистра извлекается адрес таблицы страниц (АТ)
15. Вычисляется адрес дескриптора: $a = AT + (p * L)$ - запись таблицы страниц
16. Извлекается номер физической страницы p
17. К номеру присоединяется смещение s

34. Управление памятью. Понятие памяти и внешней памяти. Оптимизация страничной памяти. Многоуровневая страничная память: принципы работы, особенности. Сегментное распределение памяти: сегменты, таблица сегментов и их содержимое, преобразование адресов, преимущества и недостатки.

Оптимизация страничной памяти:

- ❑ **MMU (Memory Management Unit)** – аппаратное преобразование адресов
- ❑ **TLB (Translation Lookaside Buffer)** – кэширование таблиц страниц
- ❑ Оптимизация размера страницы и стратегии замещения

Многоуровневая страничная память:

- ❑ Виртуальное адресное пространство делится на разделы, разделы – на страницы
- ❑ **Таблица разделов (каталог страниц)** – содержит адреса таблиц страниц разделов
- ❑ Позволяет выгружать части таблиц страниц, экономит память
- ❑ Требуется дополнительных обращений к памяти

Сегментное распределение:

- ❑ Виртуальное пространство делится на сегменты произвольного размера (по смыслу данных)
- ❑ Виртуальный адрес: (номер сегмента, смещение в сегменте)

Таблица сегментов содержит:

6. Базовый физический адрес сегмента
7. Размер сегмента
8. Правила доступа
9. Признаки модификации, присутствия, обращения

Преобразование адресов:

- ❑ По номеру сегмента находится дескриптор сегмента в таблице сегментов
- ❑ Проверяется, загружен ли сегмент в физическую память
- ❑ Если да — базовый адрес + смещение = физический адрес
- ❑ Если нет — страничная ошибка, сегмент загружается с диска

Преимущества:

- ❑ Дифференцированные права доступа к сегментам (каждый сегмент имеет свои права доступа независимо от других)
- ❑ Разделение по смысловому значению данных (память делится на сегменты по логическому назначению, а не просто по физическим блокам)

Недостатки:

- ❑ Операция сложения замедляет преобразование (vs конкатенация в страничной)
- ❑ Избыточность (сегмент > страница)
- ❑ Фрагментация из-за непредсказуемых размеров

35. Управление памятью. Понятие памяти и внешней памяти. Сегментно-страничное распределение памяти: сегменты и страницы, способы организации адресного пространства, способы адресации, алгоритмы преобразования адресов для каждого способа. Разделяемые сегменты.

Память:

2. **Память** – оперативная память (RAM), требует электропитания
3. **Внешняя память** – жёсткий диск, файл подкачки (swap file)

Сегментно-страничное распределение:

Комбинация сегментного и страничного механизмов. Виртуальное пространство делится на сегменты (логические единицы), каждый сегмент делится на страницы (фиксированного размера).

Сегменты:

- ☐ Логические единицы программы (код, данные, стек)
- ☐ Произвольного размера
- ☐ Имеют смысловое значение
- ☐ Разные права доступа для разных сегментов

Страницы:

8. Части сегментов фиксированного размера (например, 4 КБ)
9. Единицы перемещения между RAM и диском
10. Упрощают управление памятью

Способы организации адресного пространства:

Способ 1: Непрерывное линейное виртуальное адресное пространство

9. Все сегменты образуют одно непрерывное виртуальное адресное пространство
10. Сегменты размещаются последовательно друг за другом
11. Виртуальный адрес: (номер сегмента, смещение в сегменте)

Способ 2: Нумерация страниц в пределах сегмента

- ☐ Каждый сегмент имеет свою нумерацию страниц (начинается с 0)
- ☐ Сегменты независимы друг от друга
- ☐ Виртуальный адрес: (номер сегмента, номер страницы в сегменте, смещение в странице)

Способы адресации:

1. Сегментная адресация:

- 5. Адрес задаётся номером сегмента и смещением в сегменте
- 6. Соответствует логической структуре программы

2. Страничная адресация:

- ☐ Адрес задаётся номером страницы и смещением в странице
- ☐ Используется для физического размещения

3. Сегментно-страничная адресация:

- ☐ Комбинация: сначала сегмент, затем страница внутри сегмента
- ☐ Двухуровневая адресация

Алгоритмы преобразования адресов:

Способ 1: Непрерывное линейное пространство

Алгоритм:

- ☐ По номеру сегмента из таблицы сегментов получаем базовый адрес сегмента в виртуальном пространстве
- ☐ Базовый адрес + смещение = линейный виртуальный адрес
- ☐ Линейный адрес делится на номер страницы и смещение в странице
- ☐ По номеру страницы из таблицы страниц получаем номер физического кадра
- ☐ Номер физического кадра + смещение = физический адрес

Пример:

- 7. Виртуальный адрес: (сегмент 2, смещение 0x1234)
- 8. Таблица сегментов: сегмент 2 начинается с адреса 0x10000
- 9. Линейный адрес: $0x10000 + 0x1234 = 0x11234$
- 10. Номер страницы: $0x11234 / 4096 = 4$, смещение: 0x234
- 11. Таблица страниц: страница 4 → кадр 100
- 12. Физический адрес: кадр 100 * 4096 + 0x234 = 0x190234

Способ 2: Нумерация страниц в пределах сегмента

Алгоритм:

3. По номеру сегмента из таблицы сегментов получаем адрес таблицы страниц для этого сегмента
4. По номеру страницы в сегменте из таблицы страниц сегмента получаем номер физического кадра
5. Номер физического кадра + смещение в странице = физический адрес

Пример:

4. Виртуальный адрес: (сегмент 2, страница 3, смещение 0x234)
5. Таблица сегментов: сегмент 2 → таблица страниц по адресу 0x5000
6. Таблица страниц сегмента 2: страница 3 → кадр 100
7. Физический адрес: кадр 100 * 4096 + 0x234 = 0x190234

Разделяемые сегменты:

5. Несколько процессов могут использовать один и тот же сегмент
6. Дескриптор сегмента содержит поле: разделяемый/индивидуальный
7. ОС отслеживает количество процессов, использующих сегмент (счётчик ссылок)
8. При выгрузке разделяемого сегмента проверяется счётчик: если > 1, сегмент не выгружается
9. При завершении процесса счётчик уменьшается, при 0 сегмент может быть выгружен
10. Примеры: код библиотек, общие данные
11. Преимущества: экономия памяти, синхронизация данных между процессами

36. Управление памятью. Виртуальная память. Стратегии выборки страниц. Стратегии размещения: виды, преимущества и недостатки. Стратегии замещения: принцип локальности, блокировка кадров, оптимальный алгоритм, преимущества и недостатки.

Стратегии выборки страниц:

1. Выборка по требованию:

- 12. Страница загружается только при обращении к ней
- 13. Принцип локальности снижает частоту прерываний

2. Предварительная выборка:

- 13. Загружается несколько последовательных страниц сразу
- 14. Эффективна при последовательном доступе

Стратегии размещения:

1. Метод первого подходящего:

- ☐ Выбирает первый достаточный по размеру блок (80кб пропускает 50кб и выбирает 200 кб хотя дальше был блок 100 кб)
- ☐ **Преимущества:** Простота, скорость
- ☐ **Недостатки:** Засоряет начало памяти

2. Метод следующего подходящего:

- ☐ Начинает с места последнего выделения
- ☐ **Недостатки:** Хуже результаты, чаще требуется уплотнение

3. Метод наилучшего подходящего:

- ☐ Выбирает блок, наиболее близкий по размеру
- ☐ **Недостатки:** Оставляет множество мелких блоков (фрагментация), частое уплотнение

Стратегии замещения:

Принцип локальности:

Страницы, к которым недавно были обращения, вероятно будут использоваться в ближайшее время.

Блокировка кадров:

Некоторые кадры заблокированы (ядро ОС, критические данные).
Блокированная страница не может быть замещена.

Оптимальный алгоритм:

Выбор страницы, обращение к которой будет через наибольший промежуток времени.

☐ **Преимущества:** Минимальное количество прерываний, стандарт для сравнения

☐ **Недостатки:** Невозможно реализовать (требует знания будущего)

37. Управление памятью. Виртуальная память. Стратегии замещения: алгоритм дольше всех неиспользовавшегося элемента, алгоритм «первым вошел – первым вышел», часовой алгоритм, преимущества и недостатки, резидентное множество. Области видимости. Стратегии очистки.

Стратегии замещения:

1. LRU (Least Recently Used):

- ? Замещает страницу, к которой дольше всех не было обращений
- ? **Преимущества:** Близок к оптимальному
- ? **Недостатки:** Сложность реализации, большие накладные расходы

2. FIFO (First In First Out):

- ? Замещает страницу, находящуюся в памяти дольше других
- ? **Преимущества:** Простота реализации
- ? **Недостатки:** Редко даёт хорошие результаты

3. Часовой алгоритм (Clock):

- ? Использует бит использования для каждой страницы (замещается с битом использование = 0)
- ? Указатель циклически проходит по кадрам
- ? **Преимущества:** Компромисс между производительностью и накладными расходами
- ? **Недостатки:** Может быть менее эффективен, чем LRU

Резидентное множество (рабочее множество):

Часть процесса, находящаяся в основной памяти. Размер влияет на частоту прерываний и уровень мультипрограммирования.

Области видимости:

- ? **Локальная:** Выбор страницы только среди страниц процесса
- ? **Глобальная:** Выбор среди всех незаблокированных страниц в памяти

Стратегии очистки:

11. **Очистка по требованию:** Страница записывается на диск только когда она нужна для замещения другой страницы (происходит подмена модифицированной страницы (dirty = 1) на страницу, которую нужно загрузить.)

12. **Предварительная очистка:** Модифицированные страницы записываются на диск заранее, до замещения

38. Кэширование данных. Понятие кэша и кэширования. Цель кэширования. Свойство прозрачности. Иерархия памяти. Принципы действия кэш-памяти: понятие записи, обращение к данным и варианты развития событий. Свойства локальности. Проблема согласования данных.

Понятие кэша:

Кэширование – метод ускорения доступа к данным путём копирования часто используемой информации из медленной памяти в быструю (кэш).

Цель: Ускорение доступа к данным.

Свойство прозрачности:

Кэширование прозрачно для программ и пользователей. Всё выполняется автоматически.

Иерархия памяти:

16. Внешняя память (диск) – большой объём, медленный доступ (мс)
17. Оперативная память (DRAM) – средний объём, средний доступ (10-20 нс)
18. Кэш-память (SRAM) – малый объём, быстрый доступ (<8 нс)
19. Регистры процессора – минимальный объём, самый быстрый доступ (2-3 нс)

Принципы действия кэш-памяти:

Запись в кэше содержит:

18. Значение элемента данных
19. Адрес в основной памяти
20. Признак модификации и присутствия

Обращение к данным:

18. **Кэш-попадание (cache-hit):** Данные найдены в кэше, чтение из кэша
19. **Кэш-промах (cache-miss):** Данные отсутствуют, чтение из основной памяти с копированием в кэш

Свойства локальности:

14. **Временная:** Обращение по адресу → вероятно повторное обращение
15. **Пространственная:** Обращение по адресу → вероятно обращение к соседним адресам

Проблема согласования данных:

Две копии данных (в памяти и кэше) должны быть согласованы (например при изменении данных в основной памяти).

Решения:

18. **Write-through (сквозная запись):** Запись одновременно в кэш и память
19. **Write-back (обратная запись):** Запись только в кэш, с установкой признака модификации

39. Кэширование данных. Понятие кэша и кэширования. Отображение основной памяти на кэш: схемы отображения, основные понятия, особенности, смешанный подход.

Схемы отображения:

1. Случайное отображение:

- ☐ Элемент памяти может быть размещён в произвольном месте кэша
- ☐ Ассоциативный поиск по тегу (адресу в памяти)
- ☐ **Особенности:** Дорого, используется при малом объёме кэша

2. Детерминированное (прямое) отображение:

- ☐ Элемент памяти всегда отображается на одно место в кэше
- ☐ Хэширование: выделение разрядов адреса как номера строки кэша (часть адреса = номер строки кэша)
- ☐ **Особенности:** Вытеснение происходит не только при отсутствии свободного места, нет выбора данных для замещения

3. Смешанный подход (set-associative):

- ☐ Адрес отображается на группу адресов кэша
- ☐ Поиск: сначала по номеру группы, затем ассоциативный поиск в группе
- ☐ Компромисс между стоимостью и производительностью

40. Кэширование данных. Понятие кэша и кэширования. Схемы выполнения запросов в системах с кэш памятью: этапы выполнения, двухуровневый кэш. Буфер ассоциативной трансляции: понятие, назначение, принцип работы.

Схемы выполнения запросов:

Двухуровневый кэш:

10. **L1** – меньший объём, большее быстродействие (сквозная запись)
11. **L2** – больший объём, меньшее быстродействие (играет роль основной памяти для L1) (обратная запись)
12. При промахе L1 → поиск в L2 → поиск в основной памяти
13. Данные копируются между уровнями

Чтение: Поиск в L1 (если есть чтение из L1 и передача данных) -> Поиск в L2 (если есть копирование в L1 и передача данных) -> Поиск в основной памяти (копирование в L1, копирование в L2, передача данных)

Запись: Поиск в L1 (если есть запись в L1 и либо write-through: запись в L2 и RAM либо write-back: установление бита модификации) -> Поиск в L2 (если есть установление бита модификации) -> Основная память (если то либо write-no-allocate: запись только в RAM либо write-allocate: загрузка блока из L2 из RAM, загрузка блока из L2 в L1, запись в L1, обновление L2 и RAM)

TLB (Translation Lookaside Buffer):

Буфер ассоциативной трансляции для кэширования пар номеров виртуальных и физических страниц.

Назначение:

Ускорение преобразования виртуальных адресов в физические при сегментно-страничной организации памяти.

Принцип работы:

- ☐ Кэширует дескрипторы страниц из таблицы страниц
- ☐ Комбинированное отображение: прямое на наборы, случайное на строки в наборе
- ☐ Поиск в TLB перед обращением к таблицам страниц
- ☐ Позволяет заменить долгую процедуру преобразования быстрым поиском

Получение линейного адреса -> извлечение номера виртуальной страницы
-> вычисление индекса набора -> проверка всех строк в наборе (если нет, то
происходит добавление в TLB путем комбинированного отображение) ->
извлекаем физический номер -> получение физического адреса: физический
номер + смещение

41. Управление файлами. Подсистема ввода-вывода: понятие, основные компоненты, задачи. Основы параллельной работы I/O-устройств и CPU. Проблема согласования скоростей обмена и кэширование данных. Разделение устройств и данных между процессами.

Подсистема ввода-вывода:

Часть ОС для организации и управления обменом данными между программами, компонентами ОС и периферийными устройствами.

Основные компоненты:

- 14. Драйверы устройств
- 15. Файловая система
- 16. Диспетчер прерываний

Задачи:

- 15. Параллельная работа I/O и CPU
- 16. Согласование скоростей и кэширование
- 17. Разделение устройств и данных
- 18. Программный интерфейс к устройствам
- 19. Поддержка драйверов
- 20. Динамическая загрузка/выгрузка драйверов
- 21. Поддержка нескольких файловых систем
- 22. Синхронные и асинхронные операции

Параллельная работа I/O и CPU:

- ❓ **Контроллер** – специализированный блок управления устройством
- ❓ **Драйвер** – программный модуль ОС для управления устройством через контроллер
- ❓ Контроллер работает автономно, посылает прерывание по завершении

Согласование скоростей (скорость работы процессора, внешней памяти и внешних устройств сильно отличаются), решения:

- ❓ Буферизация в RAM — временное хранение в быстрой памяти для согласования CPU и устройств
- ❓ Спулинг — буферизация на диске для очень медленных устройств (принтеры)
- ❓ Буферизация в контроллерах — собственная память устройств для независимой работы

❓ Дисксовый кэш — кэширование данных в RAM для ускорения доступа к диску

Разделение устройств и данных:

❓ Монопольный или совместный доступ

❓ Контроль по правам пользователя/группы

❓ Очередь заданий для последовательных устройств (принтер) - хранения заданий в спул-файле

❓ Разделение устройств и разделение данных работает через очередь заданий для последовательных устройств

42. Управление файлами. Подсистема ввода-вывода. Программный интерфейс к устройствам. Поддержка широкого спектра драйверов: суть задачи, виды интерфейсов драйверов. Динамическая загрузка и выгрузка драйверов. Поддержка файловых систем. Синхронный и асинхронный режимы.

Программный интерфейс к устройствам:

- ☐ **Файловая модель устройств** – все устройства представляются как специальные (виртуальные) файлы (содержат метаданные для драйвера)
- ☐ Унифицированный доступ через операции чтения/записи (через драйверы)

Поддержка драйверов:

- ☐ Критически важно для универсальной ОС
- ☐ **DDK (Driver Development Kit)** – для разработки драйверов

Интерфейсы драйверов:

Разработка драйверов через DDK (Driver Development Kit):

- ☐ **DKI (Driver Kernel Interface)** – с модулями ядра, строго стандартизован
- ☐ **DDI (Driver Device Interface)** – с контроллерами устройств

Динамическая загрузка/выгрузка:

- ☐ Подгрузка драйверов при подключении устройств
- ☐ Альтернатива: статическое включение в ядро (требует перекомпиляции)

Поддержка файловых систем:

- ☐ Поддержка нескольких ФС одновременно
- ☐ **VFS (Virtual File System)** – специальный слой для унификации

Режимы операций:

13. **Синхронный:** Процесс приостанавливается до завершения операции
14. **Асинхронный:** Процесс продолжает выполнение, операция выполняется параллельно

43. Управление файлами. Подсистема ввода-вывода. Многослойная модель подсистемы ввода-вывода: вертикальный и горизонтальный слои. Менеджер ввода-вывода: понятие, функции, интерфейсы. Пример работы менеджера на примере Windows.

Многослойная модель:

- 20. **Горизонтальные слои:** Нижние – индивидуальные драйверы, верхние – обобщённые интерфейсы
- 21. **Вертикальные подсистемы:** Формируются по типам устройств (диски, графика, сеть)

Менеджер ввода-вывода:

Программная оболочка подсистемы I/O, обеспечивающая 4 интерфейса:

- 21. Пользовательский интерфейс
- 22. Интерфейс с устройствами
- 23. Интерфейс с другими подсистемами ОС
- 24. Внутренний интерфейс между компонентами

Функции:

- 20. Приём запросов от процессов
- 21. Перенаправление запросов драйверам
- 22. Возврат результатов
- 23. Экранирование драйверов от аппаратуры
- 24. Стандартизация взаимодействия драйверов

Пример (Windows):

- 16. Использует **IRP (I/O Request Packet)** – пакеты запросов
- 17. Драйверы задают взаимосвязи (bindings)
- 18. Поддерживает динамическую загрузку/выгрузку

44. Управление файлами. Подсистема ввода-вывода. Понятие драйвера. Назначение драйвера. Многоуровневый драйвер: понятие, структура, взаимодействие уровней, преимущества подхода. Виды драйверов.

Понятие драйвера:

Программный модуль ядра ОС, работающий в привилегированном режиме:

20. Управляет внешним устройством через контроллер
21. Обрабатывает прерывания
22. Предоставляет логический интерфейс программисту
23. Взаимодействует с ядром через стандартизованный интерфейс

Многоуровневый драйвер:

Совокупность нескольких драйверов на разных уровнях, совместно управляющих одним физическим устройством.

Структура:

- ☐ **Низкоуровневые (аппаратные):** Управляют конкретным устройством, обрабатывают прерывания
- ☐ **Высокоуровневые:** Используют примитивные операции, реализуют сложные функции

Взаимодействие:

- ☐ Аппаратный драйвер запускает операции и реагирует на прерывания
- ☐ Высокоуровневые драйверы вызываются аппаратными или драйверами вышележащего уровня
- ☐ Все взаимодействуют через менеджер I/O

Преимущества:

- ☐ Выбор нужного набора функций
- ☐ Несколько логических моделей одного устройства
- ☐ Расширяемость
- ☐ Упрощение разработки

Виды драйверов:

14. **Блок-ориентированные:** Работа с блоками фиксированного размера (диски)
15. **Байт-ориентированные:** Работа с потоками байтов (терминалы, принтеры)

45. Управление файлами. Файл: понятие, цели использования. Файловая система: понятие, назначение, задачи. Типы файлов. Виды иерархической структуры файловой системы. Имена файлов и их виды. Соотношения между файлами и количеством имён.

Файл:

Именованная область внешней памяти для записи и чтения данных, а также хранимые данные и набор атрибутов. Файл – абстракция над данными.

Цели использования:

- ☐ Долговременное и надёжное хранение информации
- ☐ Совместное использование информации

Файловая система:

Часть ОС, включающая:

- ☐ Совокупность всех файлов на диске
- ☐ Структуры данных для управления (каталоги, дескрипторы, таблицы распределения)
- ☐ Средства для операций с файлами

Назначение:

- ☐ Экранирование физической организации данных
- ☐ Предоставление логической модели хранения
- ☐ Удобные команды для манипулирования файлами

Задачи:

4. В однопользовательских: именование и хранение
5. В мультипрограммных: блокировка файлов, предотвращение гонок и тупиков
6. В многопользовательских: защита файлов пользователей

Типы файлов:

- ☐ Обычные файлы
- ☐ Файлы-каталоги (директории)
- ☐ Специальные файлы (устройства)
- ☐ Именованные конвейеры
- ☐ Файлы, отображаемые на память

Иерархическая структура:

11. Одноуровневая – все файлы в одном каталоге
12. Древовидная (MS-DOS, Windows) – один файл в один каталог, корневой каталог
13. Сетевая (Unix) – файл может входить в несколько каталогов

Имена файлов:

12. Простое (короткое) – в пределах каталога
13. Полное (составное) – путь от корня
14. Относительное – относительно текущего каталога
15. Уникальное (числовой идентификатор) – для ОС (inode в Unix)

Соотношения:

- ☐ Много файлов – одно простое имя (в разных каталогах)
- ☐ Один файл – одно полное имя (в древовидной структуре)
- ☐ Один файл – много полных имён (в сетевой структуре)
- ☐ Один файл – одно уникальное имя (числовой идентификатор)

46. Управление файлами. Файл и файловая система. Понятие монтирования. Атрибуты файлов: виды, доступ, организация хранения. Логическая структура файла: логическая запись, структурированные и неструктурированные файловые системы, способы структуризации.

Монтирование:

Операция объединения файловых систем с разных логических дисков в единую файловую систему с единым деревом каталогов.

Процесс монтирования:

17. Одна ФС назначается корневой
18. В корневой ФС выбирается каталог (точка подключения)
19. Подключаемая ФС включается как поддерево
20. После монтирования нет логической разницы между корневой и смонтированной ФС

Атрибуты файлов:

Виды атрибутов:

23. Тип файла (обычный, каталог, специальный)
24. Владелец и создатель
25. Права доступа
26. Пароль для доступа
27. Временные характеристики (создание, доступ, изменение)
28. Размер файла (текущий и максимальный)
29. Признаки: только чтение (R), архивный (A), скрытый (H), системный (S), временный, двоичный/символьный
30. Признак блокировки
31. Параметры логической структуры

Доступ к атрибутам:

- ☐ Чтение любых атрибутов разрешено
- ☐ Изменение только некоторых (например, права доступа, но не дата создания)

Организация хранения:

- ☐ **В каталоге (MS-DOS)** – каталог содержит имя и все атрибуты
- ☐ **В специальных таблицах (ufs Unix)** – каталог содержит имя и ссылку на таблицу (inode)

Логическая структура файла:

Логическая запись:

Наименьший элемент данных, с которым оперирует программист при обмене с внешним устройством.

Структурированные и неструктурированные ФС:

❓ **Неструктурированные** (Unix, Windows, NetWare): ФС рассматривает файл как последовательность байтов. Приложение само интерпретирует данные, указывая смещение и количество байтов.

❓ **Структурированные** (OS/360, RSX, VMS): ФС рассматривает файл как последовательность логических записей. ФС знает структуру и может выделять записи.

Способы структуризации:

❓ **Записи фиксированной длины:**

Одинаковый размер для всего файла

Прямой и последовательный доступ

Адрес записи: $L \cdot n$ (L – длина, n – номер)

❓ **Записи переменной длины:**

Разные размеры записей

Только последовательный доступ

Неиндексированный файл

❓ **Индексированные файлы:**

Записи произвольной длины

Ключевые (индексные) поля для адресации

Индексная таблица для поиска

Прямой или индексно-последовательный доступ

47. Управление файлами. Физическая организация файловой системы. Принципы работы жёсткого диска. Основные понятия: пластина, головка, дорожка, сектор. Адресация сектора. Понятие кластера и связь с секторами. Виды форматирования. Понятие раздела. Активный раздел.

Физическая организация ФС:

Описывает принципы размещения файлов, каталогов и системной информации на диске. Логическое представление не совпадает с физическим размещением.

Принципы работы жёсткого диска:

- ❑ Состоит из одной или нескольких пластин с магнитным покрытием
- ❑ Пластины образуют пакет
- ❑ На каждой стороне пластины – концентрические кольца (дорожки)
- ❑ Головка считывает/записывает данные с дорожки
- ❑ Головка позиционируется над заданной дорожкой

Основные понятия:

- ❑ **Пластина** – диск с магнитным покрытием
- ❑ **Головка** – элемент для чтения/записи данных
- ❑ **Дорожка (track)** – концентрическое кольцо на пластине
- ❑ **Цилиндр** – совокупность дорожек одного радиуса на всех поверхностях
- ❑ **Сектор (блок)** – фрагмент дорожки, фиксированный размер (обычно 512 байт)
- ❑ Сектор – наименьшая адресуемая единица обмена данными

Адресация сектора:

Физический адрес: (номер цилиндра, номер пластины, номер сектора)

Кластер:

- 15. Наименьшая единица дискового пространства, которой оперирует ФС
- 16. Размер кратен размеру сектора
- 17. При создании файла выделяется область, кратная размеру кластера
- 18. В Unix кластер называют блоком

Виды форматирования:

22. Низкоуровневое (физическое):

Создаёт дорожки и сектора
Выполняется на заводе

Записывает идентификационную информацию
Не зависит от ОС

25. Высокоуровневое (логическое):

Выполняется под конкретную ФС

Определяет размер кластера

Записывает информацию о свободном/занятом пространстве

Записывает загрузчик ОС

Раздел:

Непрерывная часть физического диска, представляемая ОС как логическое устройство (C:, D:, /dev/sda1). Информация о разделах хранится в таблице разделов.

Логическое устройство:

Представление раздела ОС как отдельного диска. На каждом логическом устройстве – одна ФС.

Активный раздел:

Раздел, из которого считывается загрузчик ОС.

48. Управление файлами. Физическая организация файловой системы. Критерии эффективности. Основные виды физической организации файлов: принципы, адресация, преимущества и недостатки.

Критерии эффективности:

- 25. Скорость доступа к данным
- 26. Объём адресной информации файла (служебные данные для описания расположения)
- 27. Степень фрагментированности дискового пространства
- 28. Максимально возможный размер файла

Виды физической организации:

1. Непрерывное размещение:

- 19. Файлу выделяется последовательность смежных кластеров
- 20. **Адресация:** номер первого кластера + размер файла
- 21. **Преимущества:** Высокая скорость доступа, минимальный объём адресной информации, простота, неограниченный размер
- 22. **Недостатки:** Сильная фрагментация, трудно определить требуемый размер

2. Связанный список кластеров:

- 24. Файл – цепочка кластеров, в начале каждого – указатель на следующий
- 25. **Адресация:** номер первого кластера (переход по цепочке)
- 26. **Преимущества:** Нет фрагментации на уровне кластеров, динамическое увеличение, минимальный объём адресной информации
- 27. **Недостатки:** Медленный доступ к произвольному месту, часть кластера используется под адресацию

3. Связанный список индексов (FAT):

- ☐ Цепочка кластеров, адресная информация в таблице FAT
- ☐ **Адресация:** через таблицу FAT с индексными указателями
- ☐ **Преимущества:** Нет фрагментации на уровне кластеров, данные заполняют кластеры (степень двойки), быстрее доступа, чем цепочка указателей
- ☐ **Недостатки:** Потери в последнем кластере, низкая надёжность, объём служебной информации зависит от размера раздела

4. Перечисление номеров кластеров:

- ❑ Простое перечисление всех номеров кластеров файла
- ❑ **Адресация:** набор номеров кластеров (прямая адресация)
- ❑ **Преимущества:** Высокая скорость доступа, нет фрагментации на уровне кластеров
- ❑ **Недостатки:** Объём адресной информации пропорционален размеру файла

5. Комбинированная схема (s5, ufs):

- ❑ Сочетание прямой и косвенной адресации
- ❑ Адресная информация в индексном дескрипторе (inode)
- ❑ **Прямая адресация:** номера кластеров хранятся непосредственно в inode
- ❑ **Косвенная адресация:** поле адреса указывает на кластер со списком номеров кластеров
- ❑ **Преимущества:** Поддержка очень больших файлов, небольшой объём адресной информации, высокая масштабируемость
- ❑ **Недостатки:** Усложнение структуры, увеличение обращений к диску для больших файлов

49. Управление файлами. Физическая организация файловой системы на примерах систем FAT, ufs и s5. Основные структурные элементы и их назначение, особенности.

Файловая система FAT:

Структурные элементы:

16. **Загрузочный сектор** – программа начальной загрузки ОС
17. **Основная копия FAT** – информация о размещении файлов (индексные указатели)
18. **Резервная копия FAT** – синхронизируется с основной, используется при повреждении
19. **Корневой каталог** – фиксированная область (32 сектора, 512 записей по 32 байта)
20. **Область данных** – хранение файлов и подкаталогов (выделение кластерами)

Адресация:

Номер первого кластера в каталоге → таблица FAT → цепочка индексных указателей → кластеры файла

Особенности:

- ☐ Простая структура
- ☐ Нет фрагментации на уровне кластеров
- ☐ Потери в последнем кластере
- ☐ Низкая надёжность
- ☐ Поддержка коротких (8.3) и длинных имён (до 255)
- ☐ Разновидности: FAT12, FAT16, FAT32 (разрядность указателей)

Файловые системы s5 и ufs:

Структурные элементы s5:

- ☐ **Загрузочный блок** – при запуске системы
- ☐ **Суперблок** – общая информация о ФС (размер, число inode, списки свободных блоков)
- ☐ **Область индексных дескрипторов (inode list)** – индексные дескрипторы файлов
- ☐ **Область данных** – обычные файлы и каталоги

Индексный дескриптор (inode):

- ❑ Фиксированный размер (64 байта в s5)
- ❑ Содержит: владелец, тип файла, права доступа, временные характеристики, число ссылок, адресную информацию, размер файла
- ❑ Номер inode – уникальное имя файла

Каталоги:

Запись содержит только символьное имя и номер inode. Имена отделены от характеристик.

Особенности ufs (развитие s5):

7. Раздел состоит из повторяющихся групп областей
8. Резервные копии суперблока (повышение надёжности)
9. Группировка по цилиндрам (уменьшение перемещений головок)
10. Поддержка длинных имён (до 255 символов)
11. Комбинированная схема адресации

Сравнение:

- ❑ **FAT:** простая структура, единая таблица, низкая надёжность, высокая фрагментация
- ❑ **s5:** отделение имени от характеристик, использование inode, ограниченные размеры
- ❑ **ufs:** развитие s5, высокая надёжность, поддержка больших файлов и дисков, оптимизация доступа

50. Управление файлами. Операции над файлами. Файловые системы с запоминанием и без запоминания состояния операций. Механизм открытия файла на примере Unix: алгоритм работы, системные структуры и их взаимосвязь. Обмен данными с файлами. Блокирование файлов.

Операции над файлами:

ФС предоставляет набор операций в виде системных вызовов:

14. creat / CreateFile – создание
15. open / CreateFile – открытие
16. read / ReadFile – чтение
17. write / WriteFile – запись
18. close / CloseHandle – закрытие
19. delete / DeleteFile – удаление
20. lseek / SetFilePointer – позиционирование
21. flock / LockFile – блокирование
22. И другие операции с каталогами и атрибутами

Файловые системы с запоминанием и без запоминания состояния:

Универсальные действия любой операции:

16. Поиск характеристик файла по символьному имени
17. Копирование характеристик с диска в память
18. Проверка прав пользователя
19. Очистка временных структур

Stateless (без запоминания состояния):

- ? Каждая операция выполняется независимо
- ? ФС не хранит информацию о предыдущих операциях
- ? Каждая операция заново выполняет все универсальные действия
- ? **Преимущества:** Высокая устойчивость к сбоям
- ? **Недостатки:** Низкая производительность
- ? **Применение:** Сетевые ФС (NFS)

Stateful (с запоминанием состояния):

7. Универсальные действия выполняются один раз в начале
8. Результаты сохраняются и используются повторно
9. Используется механизм open / close
10. **Преимущества:** Высокая производительность

11. **Недостатки:** Меньшая устойчивость к сбоям
12. **Применение:** Большинство современных ФС

Механизм открытия файла в Unix:

Алгоритм open():

- ? Преобразование символьного имени в номер inode
- ? Считывание inode с диска в структуру vnode
- ? Проверка прав доступа и создание структуры file
- ? Создание и настройка системных структур
- ? Возврат дескриптора файла (номер записи в таблице открытых файлов процесса)

Системные структуры:

vnode (virtual node):

- ? Создаётся в системной области памяти
- ? Содержит копию inode
- ? Создаётся одна на файл (независимо от числа процессов)
- ? Содержит: признаки блокировки, признаки изменения, номер устройства, номер inode, счётчик ссылок
- ? При повторном открытии счётчик ссылок увеличивается

file:

- ? Создаётся для каждого открытия файла процессом
- ? Хранит индивидуальные характеристики операций
- ? Содержит: режим открытия, указатель на vnode, текущее смещение (offset), счётчик ссылок, указатели на соседние структуры file

Таблица открытых файлов процесса:

13. Находится в контексте процесса
14. Хранит указатели на структуры file
15. Индекс записи – это дескриптор файла

Дескриптор файла:

Временное уникальное имя конкретной последовательности операций (от open до close), имеет локальное значение только для процесса.

Обмен данными с файлами:

Операции чтения/записи для открытого файла. По умолчанию синхронные (процесс блокируется до завершения операции).

Блокирование файлов:

Механизм предотвращения одновременного доступа нескольких процессов к одному файлу или его части. Функции: flock (POSIX), LockFile (Windows).